Veri Mühendisliği: Alanlar ve Ortak İlkeler

ÖFY

October 2026

Özet

Veriyle çalışan alanlar, araçlar bir kenara bırakıldığında ortak bir düşünce çekirdeğinde birleşir: veri dünyanın kendisi değil, belirli bir amaçla ve belirli bir yöntemle alınmış eksik bir parçasıdır. Bir analizin ya da modelin değeri, doğru bir karara bağlanmasından, belirsizliğini açıkça bildirmesinden ve daha önce görmediği durumlarda da doğru çalışmasından gelir. Mühendislik tarafında asıl zorluk model üretmek değil, modeli zaman içinde güvenilir, izlenebilir ve geri alınabilir biçimde çalıştırmaktır; çünkü dünya değişir ve her model, eğitildiği dünyayı yansıtmaya devam ettiği için zamanla eskir. Analitik tarafında değer, bir kurumdaki herkesin aynı tanımla aynı rakama güvenmesiyle oluşur; bu güven testlerle, dokümantasyonla, verinin kökeninin izlenmesiyle ve her temel varlık için tek bir doğru kayıt tutulmasıyla kurulur. Ölçek büyüdükçe problem hesaplamadan veriyi taşımaya ve bölmeye kayar; bulut ortamında ise her kaynak ölçülüp faturalandığı için verimli çalışmak doğrudan maliyet tasarrufu anlamına gelir. Neyin ölçüldüğü, insanların ve sistemlerin nasıl davranacağını belirler; yanlış seçilmiş bir metrik, doğru görünen ama zararlı sonuçlar üretir. Basit yöntemler çoğu durumda karmaşık olanlarla yarışabilir; karmaşıklık ancak gerçek bir ihtiyaç kanıtlandığında eklenmelidir.

Giriş

Veriyle çalışmak, tek bir meslek değil birbirine bağlı alanlardan oluşan bir ailedir. Bu ailenin başlıca üyeleri şunlardır:

- **Veri bilimi** (*data science*): veriden tahmin, açıklama ve karar desteği üretme işi.
- Makine öğrenmesi mühendisliği (*machine learning engineering*): modelleri gerçek sistemlerde çalışır hâle getirme işi.
- MLOps: modellerin canlıya alınmasını, sürümlenmesini ve izlenmesini düzenleyen pratikler bütünü.
- Analitik mühendisliği (*analytics engineering*): ham veriyi herkesin güvenip kullanabileceği temiz tablolara dönüştürme işi.
- İş zekâsı (*business intelligence*, BI): veriyi raporlara ve gösterge panellerine dönüştürerek karar vericilere sunma işi.
- Büyük veri (*big data*): tek bir bilgisayara sığmayan veriyle çalışma yöntemleri.
- Bulut veri platformları: Snowflake, Databricks, BigQuery, Redshift gibi hizmet olarak sunulan veri altyapıları.
- Veritabanı yönetimi (*database administration*, DBA): veritabanlarının kurulumu, güvenliği, performansı ve sürekliliği.
- Veri mimarisi (*data architecture*): bir kurumun bütün veri sisteminin genel tasarımı.
- Veri yönetişimi ve veri kalitesi (*data governance*, *data quality*): verinin sahipliği, doğruluğu, güvenliği ve yasal uyumu.
- Veri görselleştirme (*data visualization*): veriyi anlaşılır grafiklere dönüştürme işi.
- Ürün analitiği (*product analytics*): kullanıcıların bir üründeki davranışlarını ölçme ve yorumlama işi.
- Zaman serileri ve tahmin (*time series*, *forecasting*): zamana bağlı veriden geleceğe dair tahmin üretme işi.
- Coğrafi veri (*geospatial data*): konum bilgisi taşıyan veriyle çalışma yöntemleri.

Bu alanlar genellikle araçlar üzerinden öğretilir: Python, SQL, Spark, Power BI ve benzerleri. Ancak araçlar hızla değişir ve yapay zekâ destekli yardımcılar sayesinde araç kullanımını öğrenmek giderek kolaylaşır. Kalıcı olan ve kolayca devredilemeyen şey, her alanın taşıdığı düşünme biçimidir: hangi soruların sorulması gerektiği, hangi tuzakların bulunduğu ve hangi ödünleşimlerin (*trade-off*) kaçınılmaz olduğu. Yaygın ama yanlış kabul, veri işinin esas olarak araç bilgisi

olduğudur. Oysa doğru aracı kullanan ama yanlış soruyu soran, yanlılığı fark etmeyen ya da metriği yanlış seçen bir çalışma, teknik olarak kusursuz olsa bile yanlış sonuç üretir.

Bu not her alanın temel konu başlıklarını ve bu başlıklardan çıkan dersleri somut örneklerle ele alır. Doğal dil işleme (*natural language processing*, NLP) ve üretken yapay zekâ (*generative AI*) kapsam dışındadır.

Veri Biliminin Düşünce İlkeleri

Veri biliminin araçlardan bağımsız çekirdeği, belirsizlik altında doğru çıkarım yapmayı sağlayan bir dizi ilkeden oluşur. Bu ilkeler aşağıda altı grupta toplanmıştır.

Soru ve Bağlam

Soru sormak

Veri işinin büyük kısmı doğru soruyu bulmaktır. "Veride ne var?" sorusuyla başlayan bir analiz, belirli bir karara bağlanmayan çok sayıda rastgele bulgu üretir. "Hangi kararı vermeye çalışıyoruz?" sorusuyla başlayan bir analiz ise hangi verinin gerektiğini, hangi kesinliğin yeterli olduğunu ve sonucun nasıl kullanılacağını baştan belirler.

Örneğin bir market zinciri veri ekibinden "verimize bir bakın" diye istediğinde yüzlerce grafik ortaya çıkar ve hiçbiri bir eyleme dönüşmez. Soru "Hangi şubede hangi ürünün sipariş miktarı azaltılırsa bozulan ürün israfı düşer?" biçiminde kurulduğunda ise analiz doğrudan bir sipariş kararına bağlanır.

Alan bilgisi

Veriyi doğru yorumlamak için verinin üretildiği işin kendisini bilmek gerekir. Sayılar, onları üreten süreç bilinmeden okunduğunda yanıltıcı olabilir.

Bu ilkenin bilinen bir örneği tıp alanından gelir. Zatürre hastalarının ölüm riskini tahmin etmek için geliştirilen bir model, astımı olan hastaların daha düşük risk taşıdığını öğrenmiş ve bu hastaların taburcu edilebileceği sonucuna yaklaşmıştır (Caruana vd., 2015). Gerçekte astımlı zatürre hastaları yüksek risk taşıdıkları için doğrudan yoğun bakıma alınıyor ve daha yoğun tedavi görüyordu. Düşük ölüm oranlarının nedeni astımın koruyucu olması değil, gördükleri tedaviydi. Veri doğruydu; yorum yanlıştı. Bu hatayı fark etmek için istatistik değil, hastanenin nasıl çalıştığını bilmek gerekiyordu.

Maliyet ve fayda

Her hata aynı bedeli taşımaz. Bir modelin ya da kararın değerlendirilmesinde yalnızca kaç kez yanıldığına değil, hangi yönde yanıldığına da bakılmalıdır. İki temel hata türü vardır: var olan bir şeyi kaçırmak (yanlış negatif) ve olmayan bir şeyi varmış gibi göstermek (yanlış pozitif).

Örneğin bir kanser tarama testinde hasta bir kişiye "sağlıklısınız" demek, sağlıklı bir kişiye "ileri tetkik gerekiyor" demekten çok daha pahalıdır; ilki tedavinin gecikmesine, ikincisi ise geçici bir kaygıya ve ek bir teste yol açar. Bir istenmeyen e-posta (*spam*) filtresinde ise durum tersidir: önemli bir iş e-postasını yanlışlıkla çöpe atmak, bir reklam e-postasını gelen kutusunda bırakmaktan daha zararlıdır. Aynı doğruluk oranına sahip iki model, hatalarının yönüne göre birinde kabul edilebilir, ötekinde kabul edilemez olabilir.

Belirsizlik ve Olasılık

Belirsizlikle düşünmek

Hiçbir veri temelli sonuç kesin değildir. Her tahmin ve her ölçüm bir hata payı taşır. Bu payı ölçmek ve açıkça bildirmek, sonucun kendisi kadar önemlidir; hata payı belirtilmeyen bir sonuç, olduğundan daha kesin görünür.

Örneğin hava durumu tahmininde "yarın %70 yağmur ihtimali" ifadesi, yağmurun kesin yağacağı anlamına gelmez; benzer koşullardaki on günün yaklaşık yedisinde yağmur yağdığını bildirir. Seçim anketlerinde verilen "±3 puan" hata payı da aynı şeyi ifade eder: %48 ile %51 alan iki aday arasındaki fark, hata payının içinde kaldığı için anlamlı bir öndelik göstermeyebilir.

Bayesçi ve frekansçı bakış

Olasılığı anlamanın iki ana yaklaşımı vardır. **Frekansçı yaklaşım** (*frequentist*), olasılığı bir olayın çok sayıda tekrar edildiğinde gerçekleşme sıklığı olarak tanımlar. Bayesçi yaklaşım (*Bayesian*) ise olasılığı bir inanç derecesi olarak görür ve bu inancın yeni veri geldikçe güncellenmesini öngörür. Bayesçi bakışın öğrettiği temel ders, yeni bir kanıtı değerlendirirken önceki bilginin (*prior*) hesaba katılması gerektiğidir.

Bunun klasik örneği nadir hastalık testidir. Bir hastalık 1000 kişide 1 kişide görülsün ve bu hastalığı tespit eden test %99 doğru olsun; yani hastaların %99'unu yakalasın ve sağlıklıların %99'una doğru biçimde negatif sonuç versin. Bir kişinin testi pozitif çıktığında çoğu insan hasta olma ihtimalinin %99 olduğunu düşünür. Hesap farklı bir sonuç verir. 100.000 kişilik bir toplulukta 100 hasta vardır; test bunların 99'unu yakalar. Kalan 99.900 sağlıklı kişinin %1'i, yani yaklaşık 999 kişi de yanlışlıkla pozitif sonuç alır. Toplam yaklaşık 1098 pozitif sonucun yalnızca 99'u gerçek hastadır:

$$\frac{99}{99 + 999} \approx 0{,}09$$

Yani pozitif sonuç alan bir kişinin gerçekten hasta olma ihtimali yaklaşık %9'dur. Bu sonuç, hastalığın nadirliği (önceki bilgi) hesaba katıldığında ortaya çıkar. Önceki bilginin göz ardı edilmesine taban oranı yanılgısı (*base rate fallacy*) denir.

Sinyal ve gürültü

Verideki her dalgalanma anlamlı değildir. Bir serideki değişimin bir kısmı gerçek bir etkiden (sinyal), bir kısmı ise rastlantısal oynamadan (gürültü) kaynaklanır. Gürültüye bakarak karar vermek, olmayan bir şeye tepki vermek demektir.

Örneğin bir mağazanın satışları bir hafta %5 düştü diye yeni bir kampanya başlatılırsa ve bu düşüş rastlantısal bir oynamaysa, kampanya gereksiz bir maliyettir. Ertesi hafta satışların kendiliğinden toparlanması da kampanyanın işe yaradığı yanılgısını doğurur. Bir değişimin gerçek olup olmadığını anlamak için önce olağan oynamanın ne kadar büyük olduğunun bilinmesi gerekir.

Ortalamaya doğru gerileme

Aşırı iyi ya da aşırı kötü bir sonuç, büyük olasılıkla bir sonraki ölçümde ortalamaya yaklaşır. Bunun nedeni, uç sonuçların genellikle gerçek bir düzey ile o anki şansın toplamından oluşmasıdır; şans bir sonraki seferde aynı yönde tekrarlanmaz. Bu olgu ilk kez Galton (1886) tarafından boy ölçümleri üzerinde tanımlanmıştır.

Kahneman (2011), İsrail Hava Kuvvetleri'ndeki uçuş eğitmenlerinin şu gözlemini aktarır: iyi bir iniş için övülen öğrenciler bir sonraki denemede daha kötü, kötü bir iniş için azarlanan öğrenciler ise daha iyi performans gösteriyordu. Eğitmenler bundan azarın işe yaradığı, övgünün ise zarar verdiği sonucunu çıkarmıştı. Gerçekte övgü de azar da etkisizdi; uç performanslar kendiliğinden ortalamaya dönüyordu. Aynı yanılgı, kötü bir çeyrekten sonra yönetimin değiştirilmesi ve ardından gelen toparlanmanın yeni yönetime atfedilmesinde de görülür.

Nedensellik ve Yanlılık

Korelasyon ve nedensellik

İki değişkenin birlikte değişmesi (korelasyon), birinin ötekine neden olduğunu (nedensellik) göstermez. Birlikte değişimin arkasında çoğu zaman ikisini birden etkileyen üçüncü bir değişken bulunur. Bu değişkene **karıştırıcı değişken** (*confounding variable*) denir.

Örneğin yaz aylarında dondurma satışları arttıkça boğulma vakaları da artar. Dondurma boğulmaya neden olmaz; sıcak hava hem dondurma tüketimini hem de denize girenlerin sayısını artırır. Nedenselliği göstermek için ya kontrollü bir deney yapılması ya da karıştırıcı değişkenlerin dikkatle hesaba katılması gerekir.

Örneklem ve yanlılık

Veri, dünyanın kendisi değil ondan alınmış bir parçadır. Bu parçanın nasıl seçildiği sonucu doğrudan belirler. Hangi durumların veride bulunduğu kadar hangilerinin bulunmadığı da sorgulanmalıdır. Yalnızca belirli bir eleme sürecinden geçmiş örneklere bakılmasından doğan hataya **hayatta kalan yanlılığı** (*survivorship bias*) denir.

Bunun en bilinen örneği İkinci Dünya Savaşı'ndan gelir. Görevden dönen savaş uçaklarındaki kurşun izleri incelenmiş ve en çok vurulan bölgelerin zırhla güçlendirilmesi düşünülmüştü. İstatistikçi Abraham Wald, verinin yalnızca geri dönebilen uçaklardan geldiğine dikkat çekti (Mangel ve Samaniego, 1984). Hiç kurşun izi görülmeyen bölgelerden vurulan uçaklar düşmüş ve geri dönememişti; dolayısıyla asıl güçlendirilmesi gereken yerler, dönen uçaklarda iz bulunmayan bölgelerdi. Benzer biçimde yalnızca başarılı girişimcilerin hikâyelerine bakarak başarının formülünü çıkarmaya çalışmak, aynı yolu izleyip batan binlerce girişimi görmezden gelir.

Simpson paradoksu

Gruplara ayrı ayrı bakıldığında görülen bir eğilim, gruplar birleştirildiğinde tersine dönebilir. Bu olgu Simpson (1951) tarafından tanımlanmıştır ve genellikle grupların büyüklüklerinin ya da zorluk derecelerinin farklı olmasından kaynaklanır.

Charig vd. (1986), böbrek taşı tedavisinde iki yöntemi karşılaştırmıştır. Sonuçlar şöyledir:

	Yöntem A	Yöntem B
Küçük taşlar	%93 (81/87)	%87 (234/270)
Büyük taşlar	%73 (192/263)	%69 (55/80)
Toplam	%78 (273/350)	%83 (289/350)

Yöntem A hem küçük hem büyük taşlarda daha başarılıdır, ancak toplamda Yöntem B daha başarılı görünür. Bunun nedeni, Yöntem A'nın ağırlıklı olarak tedavisi daha zor olan büyük taşlarda, Yöntem B'nin ise ağırlıklı olarak kolay olan küçük taşlarda uygulanmış olmasıdır. Ders şudur: toplu bir rakam, altındaki grupların bileşimi bilinmeden yorumlanmamalıdır.

Deney tasarımı

Bir değişikliğin gerçekten bir etkisi olup olmadığını anlamanın en güvenilir yolu kontrollü deneydir. Kontrollü deneyde katılımcılar rastgele iki gruba ayrılır; bir gruba değişiklik uygulanır, diğerine uygulanmaz. Rastgele atama, iki grup arasındaki tek sistematik farkın değişikliğin kendisi olmasını sağlar; böylece karıştırıcı değişkenlerin etkisi ortadan kalkar.

Örneğin bir e-ticaret sitesi "Satın Al" düğmesinin rengini değiştirmek istediğinde, kullanıcıların yarısına rastgele yeni rengi, diğer yarısına eski rengi gösterir. Satın alma oranında yeterince büyük ve istatistiksel olarak anlamlı bir fark ortaya çıkarsa, bu farkın nedeninin düğme olduğu söylenebilir. Bu tür deneylere A/B testi denir.

Çoklu karşılaştırma tuzağı

Yeterince çok sayıda şey test edildiğinde, bazıları yalnızca şans eseri "anlamlı" çıkar. Yaygın olarak kullanılan %5 anlamlılık düzeyinde, aslında hiçbir etkisi olmayan 20 ayrı değişiklik test edilirse, en az birinin anlamlı çıkma olasılığı şudur:

$$1 - 0{,}95^{20} \approx 0{,}64$$

Yani yaklaşık %64 ihtimalle en az bir sahte bulgu elde edilir. Anlamlı sonuç çıkana kadar farklı analizler denenip yalnızca anlamlı çıkanın raporlanmasına *p-hacking* denir (Simmons, Nelson ve Simonsohn, 2011). Bu tuzaktan korunmak için hipotezlerin analizden önce belirlenmesi ve çok sayıda test yapıldığında anlamlılık eşiğinin buna göre sıkılaştırılması gerekir.

Model ve Genelleme

Model bir basitleştirmedir

Her model, gerçekliğin bazı ayrıntılarını bilinçli olarak dışarıda bırakan bir özettir. Modelin değeri gerçeği eksiksiz yansıtmasından değil, belirli bir amaç için işe yaramasından gelir. Bu fikir istatistikçi George Box'a atfedilen "bütün modeller yanlıştır, bazıları işe yarar" sözüyle özetlenir (Box, 1976; Box ve Draper, 1987).

Metro haritası bunun iyi bir örneğidir. Haritadaki mesafeler, hatların kıvrımları ve istasyonların gerçek konumları yanlıştır. Buna karşın hangi hattan hangi istasyona gidileceğini anlamak için mükemmel çalışır; çünkü yolcu için önemli olan bağlantılardır, gerçek geometri değil. Aynı harita yürüme mesafesini hesaplamak için kullanılırsa yanıltır. Model, yapıldığı amacın dışında kullanıldığında bozulur.

Ezberleme ve genelleme

Bir model, eğitildiği veriyi çok iyi açıklayıp yeni veride başarısız olabilir. Bu duruma **aşırı öğrenme** (*overfitting*) denir. Modelin asıl hedefi eldeki veriyi açıklamak değil, görmediği veride de doğru çalışmaktır (genelleme, *generalization*).

Sınava yalnızca geçmiş yılların sorularını ezberleyerek hazırlanan bir öğrenci, aynı sorular gelirse tam puan alır; sorular biraz değiştiğinde ise başarısız olur. Aşırı öğrenen model de verideki gerçek örüntüyle birlikte rastlantısal gürültüyü de ezberlemiştir. Bu yüzden modeller her zaman eğitimde kullanılmamış ayrı bir veri üzerinde test edilir.

Basitlik ve karmaşıklık dengesi

Daha karmaşık bir model her zaman daha iyi değildir. Karmaşık modeller daha fazla veri ister, daha kolay aşırı öğrenir, anlaşılması ve bakımı daha zordur. Basit bir model çoğu zaman daha sağlam, daha hızlı ve daha açıklanabilirdir.

Netflix, 2006'da öneri sistemini %10 iyileştirecek ekibe 1 milyon dolar ödül vereceğini açıkladı ve ödül 2009'da verildi. Ancak Netflix daha sonra, kazanan çözümün tamamını hiçbir zaman üretime almadığını bildirdi (Amatriain ve Basilico, 2012). Yüzlerce modelin birleşiminden oluşan çözümün sağladığı ek doğruluk, onu gerçek sistemde çalıştırmanın mühendislik maliyetini karşılamıyordu. Daha önceki aşamalardan gelen ve çok daha basit olan iki yöntem ise kullanılmaya devam etti.

Veri sızıntısı

Model, eğitim sırasında gerçek kullanım anında bilinmeyecek bir bilgiyi görürse, sahte bir başarı gösterir. Bu duruma **veri sızıntısı** (*data leakage*) denir (Kaufman vd., 2012). Sızıntı genellikle fark edilmez, çünkü model testte olağanüstü

başarılı görünür; sorun ancak canlı kullanımda performans çöktüğünde ortaya çıkar.

Örneğin müşterilerin aboneliklerini iptal edip etmeyeceğini tahmin eden bir modelin eğitim verisinde "iptal nedeni" sütunu bulunsun. Bu sütun yalnızca iptal eden müşterilerde doludur. Model, bu sütunun dolu olup olmamasına bakarak iptali neredeyse kusursuz biçimde "tahmin eder". Oysa gerçek kullanımda, henüz iptal etmemiş bir müşteri için bu bilgi zaten yoktur; model işe yaramaz hâle gelir.

Dağılım değişimi

Dünya değişir ve onunla birlikte veri de değişir. Bir model, eğitildiği dönemin koşullarını öğrenir; koşullar değiştiğinde modelin varsayımları geçerliliğini yitirir. Bu duruma **dağılım değişimi** (*distribution shift*) denir.

2020'deki pandemi bunun en belirgin örneğidir. 2019 ve öncesinin verisiyle eğitilmiş talep tahmin modelleri, tuvalet kâğıdı, maske ve el dezenfektanı talebindeki ani sıçramaları, restoran ve seyahat talebindeki çöküşü öngöremedi. Modeller teknik olarak hatasızdı; ancak öğrendikleri dünya artık yoktu.

Geri besleme döngüsü

Bir modelin verdiği kararlar, gelecekte toplanacak veriyi etkileyebilir. Bu durumda model, kendi kararlarının sonucu olan veriyle yeniden eğitildiğinde kendini doğrulamaya başlar.

Ensign vd. (2018), tahmine dayalı polislik sistemlerinde bu döngüyü göstermiştir. Model bir bölgede suç beklediğinde oraya daha fazla devriye gönderilir. Daha fazla devriye, o bölgede daha fazla suçun gözlemlenip kaydedilmesine yol açar. Model bu yeni veriyle güncellendiğinde aynı bölgeyi daha da riskli görür. Başlangıçtaki küçük bir fark, gerçek suç oranlarından bağımsız olarak giderek büyür.

Ölçme

Doğru metriği seçmek

Neyin ölçüldüğü, neyin elde edileceğini belirler. Bir metrik hedef hâline getirildiğinde insanlar ve sistemler metriği iyileştirmeye odaklanır; metriğin temsil ettiği asıl amaç ise ikinci plana düşer. Bu olgu, Goodhart'ın (1975) ekonomi politikası üzerine yaptığı gözleme dayanır ve genellikle "bir ölçüt hedef hâline geldiğinde iyi bir ölçüt olmaktan çıkar" biçiminde ifade edilir (Strathern, 1997).

Bu ilkeye sıklıkla örnek gösterilen bir anlatıya göre, sömürge dönemi Hindistan'ında yönetim, kobra sayısını azaltmak için her ölü kobra karşılığında ödül vermiştir. İnsanlar ödül almak için kobra yetiştirmeye başlamış, program iptal edildiğinde ise değersizleşen kobraları salıvermiş ve kobra sayısı başlangıçtakinden fazla olmuştur.[1] Benzer biçimde bir çağrı merkezinde "ortalama görüşme süresi"

[1] Bu anlatı "kobra etkisi" adıyla Siebert (2001) tarafından yaygınlaştırılmıştır; tarihsel doğruluğu kesin olarak belgelenmemiştir ve bir örnek anlatı olarak değerlendirilmelidir.

metriği düşürülmeye çalışıldığında, temsilciler sorunu çözmeden görüşmeyi kapatmaya başlar ve aynı müşteri tekrar arar.

Güven ve Sorumluluk

Veri kalitesi

En iyi model bile hatalı veriyle doğru sonuç üretemez. Bu ilke "çöp girerse çöp çıkar" (*garbage in, garbage out*) ifadesiyle özetlenir. Eksik, hatalı, eskimiş ya da yanlış birimdeki veri, sonraki bütün adımları geçersiz kılar.

NASA'nın Mars Climate Orbiter uydusu 1999'da Mars yörüngesine girerken kaybolmuştur. İnceleme kurulu, uyduyu üreten firmanın yazılımının itki verilerini İngiliz ölçü biriminde (pound-force saniye) ürettiğini, NASA'nın navigasyon yazılımının ise bu verileri metrik birimde (newton saniye) beklediğini tespit etmiştir (NASA, 1999). Hesaplamalar doğruydu; yanlış olan, verinin birimiydi.

Tekrarlanabilirlik

Başka biri aynı veriyle ve aynı yöntemle aynı sonuca ulaşamıyorsa, sonuç güvenilir değildir. Tekrarlanabilirlik (*reproducibility*), bir bulgunun rastlantı ya da hata ürünü olmadığını gösterir.

Open Science Collaboration (2015), psikoloji alanında yayımlanmış 100 çalışmayı yeniden yapmış ve özgün çalışmaların büyük çoğunluğu anlamlı sonuç bildirmiş olmasına karşın, tekrar çalışmalarının yalnızca yaklaşık %36'sında anlamlı sonuç elde edebilmiştir. Bu bulgu, tek bir çalışmanın sonucuna dayanarak kesin yargıya varmanın riskini gösterir. Veri işinde tekrarlanabilirlik; kullanılan verinin, kodun ve ayarların kaydedilmesiyle sağlanır.

Açıklanabilirlik

Bir modelin doğru çalışması, özellikle insanları doğrudan etkileyen kararlarda yeterli değildir; kararın neden verildiğinin de açıklanabilmesi gerekir. Açıklanabilirlik (*explainability*) hem hataların fark edilmesini hem de kararların hesap verebilir olmasını sağlar.

Örneğin bir bankanın modeli bir kredi başvurusunu reddettiğinde "model öyle karar verdi" cevabı ne başvuru sahibi için ne de denetleyici kurumlar için yeterlidir. Avrupa Birliği Genel Veri Koruma Tüzüğü (GDPR), yalnızca otomatik işlemeye dayalı önemli kararlar konusunda kişilere çeşitli haklar tanır (Avrupa Birliği, 2016, Madde 22). Bu hükmün tam bir "açıklama hakkı" sağlayıp sağlamadığı hukuk literatüründe tartışmalıdır (Wachter, Mittelstadt ve Floridi, 2017); ancak yön, otomatik kararların gerekçelendirilebilir olması gerektiğidir.

Etik ve sorumluluk

Veri çoğunlukla insanlarla ilgilidir. Bir model geçmiş veriden öğrendiğinde, o verideki adaletsizlikleri de öğrenir ve onları otomatik, ölçeklenmiş ve görünmez bir biçimde sürdürebilir.

Reuters'ın aktardığına göre Amazon, özgeçmişleri puanlayan deneysel bir işe alım modeli geliştirmiştir (Dastin, 2018). Model, geçmiş on yılın başvurularından öğrenmişti ve bu dönemde teknik pozisyonlara işe alınanların büyük çoğunluğu erkekti. Model, "kadın" kelimesini içeren ifadeler taşıyan özgeçmişlerin puanını düşürmeye başladı. Amazon bu modeli kullanmaktan vazgeçti. Bu örnek, modelin doğruluğunun adaletin garantisi olmadığını gösterir: model geçmişi doğru biçimde öğrenmişti, ama öğrendiği geçmiş adil değildi.

Veriyle hikâye anlatmak

Bir analizin değeri, başkalarının onu anlayıp ona göre karar vermesiyle ortaya çıkar. Doğru ama anlaşılmayan bir sonuç, hiç üretilmemiş bir sonuçla aynı etkiye sahiptir. Sonucu sade, dürüst ve karar vericinin sorusuna bağlı biçimde anlatmak ayrı bir beceridir.

Florence Nightingale, Kırım Savaşı sırasında İngiliz askerlerinin büyük çoğunluğunun çatışmada değil, hastanelerdeki kötü hijyen koşulları nedeniyle önlenebilir hastalıklardan öldüğünü göstermiştir. Bu bulguyu, ölüm nedenlerini ay ay renklerle ayıran dairesel bir grafikle sunmuştur (Nightingale, 1858). Tablolarla ikna edilemeyen yetkililer bu grafikle ikna olmuş ve hastane koşullarında reformlar yapılmıştır.

Genel Ders

Veri biliminin düşünce ilkeleri tek bir noktada buluşur: veri, gerçekliğin kendisi değil, belirli bir süreçle üretilmiş eksik bir temsilidir. Bu temsilden doğru sonuç çıkarmak için verinin nasıl üretildiği (yanlılık, kalite, alan bilgisi), sonucun ne kadar kesin olduğu (belirsizlik, gürültü, çoklu karşılaştırma), ilişkinin nedensel olup olmadığı (karıştırıcı değişkenler, deney) ve modelin zaman içinde geçerliliğini koruyup korumadığı (dağılım değişimi, geri besleme) sorgulanmalıdır.

Makine Öğrenmesi Mühendisliği

Makine öğrenmesi mühendisliği, işe yarayan bir modeli gerçek bir sistemde, gerçek kullanıcılarla, sürekli ve güvenilir biçimde çalışır hâle getirme işidir. Veri bilimci "bu model doğru mu?" sorusunu sorar; makine öğrenmesi mühendisi ise "bu model yarın, bir milyon kullanıcıyla, gece saat üçte, kimse başında değilken de doğru çalışacak mı?" sorusunu sorar. Sculley vd. (2015), gerçek makine öğrenmesi sistemlerinde model kodunun bütün sistemin yalnızca küçük bir parçası olduğunu; veri toplama, doğrulama, sunma, izleme ve yapılandırma altyapısının çok daha büyük yer kapladığını göstermiştir.

Algoritmaların çalışma mantığı

Her algoritma, dünyaya dair bir varsayımla gelir. Doğrusal regresyon, değişkenler arasındaki ilişkinin düz bir çizgiyle ifade edilebileceğini varsayar. En yakın komşu (*k-nearest neighbors*) yöntemi, birbirine benzeyen örneklerin benzer sonuçlara sahip olacağını varsayar. Karar ağaçları, veriyi art arda sorulan evet-hayır sorularıyla gruplara ayırabileceğini varsayar.

Bu varsayımlar veriye uymadığında, algoritma ne kadar iyi ayarlanırsa ayarlansın yanılır. Örneğin bir evin fiyatı ile metrekaresi arasındaki ilişki belirli bir büyüklükten sonra düzleşiyorsa, düz bir çizgi bu ilişkiyi hiçbir zaman doğru yakalayamaz. Wolpert (1996), bütün problemlerde ortalama olarak diğerlerinden üstün olan tek bir öğrenme algoritması bulunmadığını göstermiştir (*no free lunch* teoremi). Ders şudur: araç değil varsayım seçilir; bir algoritmanın başarısı, varsayımının veriye ne kadar uyduğuna bağlıdır.

Derin öğrenme

Derin öğrenme (*deep learning*), çok katmanlı yapay sinir ağlarıyla verideki örüntülerin otomatik olarak öğrenilmesidir. Klasik yöntemlerde bir insanın, modele hangi özelliklere bakacağını tarif etmesi gerekir; derin öğrenmede ise yeterli veri ve hesaplama gücü sağlandığında model bu özellikleri kendisi bulur.

Örneğin bir görüntüde kedi olup olmadığını anlamak için kulak, bıyık ve göz şekillerini tarif eden kurallar yazmak hem çok zor hem de kırılgandır. Derin öğrenme modeline ise binlerce etiketli kedi fotoğrafı gösterilir ve model ayırt edici özellikleri kendi öğrenir. Bu yaklaşımın bedeli; çok büyük miktarda veri, yüksek enerji ve donanım maliyeti ve modelin kararlarının neden öyle olduğunu anlamanın güçleşmesidir. Ders şudur: bazı problemlerde kural yazmak yerine örnek göstermek daha iyi çalışır, ancak bu tercih şeffaflıktan ve kaynaktan ödün vermeyi gerektirir.

Model eğitimi ve hiperparametre ayarı

Model eğitimi, modelin veriden öğrenmesi sürecidir. Bu süreç, modelin kendisinin öğrenmediği ama eğitimden önce insan tarafından belirlenen ayarlara

bağlıdır. Bu ayarlara **hiperparametre** (*hyperparameter*) denir: öğrenme hızı, ağacın derinliği, katman sayısı gibi.

Aynı model, hiperparametrelerine göre çok iyi de çok kötü de sonuç verebilir. Örneğin bir karar ağacının derinliğine sınır konmazsa, ağaç eğitim verisindeki her örneği ayrı bir yaprakta ezberler ve aşırı öğrenir; derinlik çok sınırlanırsa veri içindeki örüntüyü yakalayamaz. Bergstra ve Bengio (2012), hiperparametre aramasında rastgele denemenin, sistematik ızgara aramasından çoğu zaman daha verimli olduğunu göstermiştir. Ders şudur: eğitim bir deneme-yanılma sürecidir; sonuç tek bir denemeye bağlanmamalı, denemeler sistemli yapılmalı ve her biri kaydedilmelidir.

Modeli bir servis olarak sunmak

Bir model, ancak başka sistemler ve insanlar onu kullanabildiğinde değer üretir. Bir analistin bilgisayarında çalışan model ile bir uygulamanın her saniye binlerce kez çağırdığı model aynı şey değildir. Modeli bir uygulama programlama arayüzü (*application programming interface*, API) arkasına koymak, onu sıradan bir yazılım bileşeni hâline getirir.

Bu noktadan sonra model, yazılımın bütün gereklerine tabi olur: hatalı girdiye nasıl tepki vereceği, zaman aşımında ne olacağı, yeni sürümün eskisinin yerini nasıl alacağı, bir sorun çıktığında nasıl geri dönüleceği. Örneğin bir öneri modeli yanıt vermediğinde uygulama tamamen çökmemeli, en çok satan ürünler gibi basit bir yedek listeye dönebilmelidir. Ders şudur: canlıdaki model bir yazılım parçasıdır ve yazılım mühendisliğinin bütün disiplinleri ona da uygulanır.

Donanım kullanımı ve performans

Gerçek sistemlerde hız ve maliyet, doğruluk kadar önemlidir. Grafik işlem birimleri (*graphics processing unit*, GPU), çok sayıda basit işlemi paralel yapabildikleri için model eğitimini ve çalıştırmayı hızlandırır, ancak pahalıdır.

Örneğin %1 daha doğru ama yanıt süresi on kat uzun olan bir model, kullanıcının anında yanıt beklediği bir arama kutusunda kullanılamaz. Benzer biçimde her tahmin için pahalı bir GPU gerektiren bir model, düşük kârlı bir işte kendi maliyetini karşılayamayabilir. Ders şudur: doğruluk, hız ve maliyet arasında her zaman bir denge vardır ve en doğru model her zaman en iyi model değildir.

Büyük veriyle model eğitimi

Veri büyüdükçe model eğitimi bir matematik probleminden bir mühendislik problemine dönüşür. Veri tek bir makinenin belleğine sığmadığında; verinin parçalara bölünmesi, makineler arasında taşınması ve eğitimin birden fazla makineye dağıtılması gerekir.

Örneğin bir dizüstü bilgisayarda birkaç dakikada eğitilen bir model, verinin yüz katına çıkması durumunda aynı yöntemle günler sürebilir ya da bellek yeter-

sizliği nedeniyle hiç çalışmayabilir. Küçük ölçekte önemsiz olan veri okuma hızı, ağ trafiği ve makineler arası eşgüdüm, büyük ölçekte asıl darboğaz hâline gelir. Ders şudur: ölçek her şeyi değiştirir; küçük ölçekte çalışan bir yöntem büyük ölçekte çökebilir.

Genel Ders

İşe yarayan bir model bulmak işin başlangıcıdır. Asıl iş, o modeli her gün, herkes için, beklenmedik durumlarda da sorunsuz çalışır hâle getirmektir. Bunun için algoritmanın varsayımlarının bilinmesi, eğitimin sistemli yürütülmesi, modelin bir yazılım bileşeni gibi ele alınması ve doğruluk, hız ve maliyet arasındaki dengenin bilinçli kurulması gerekir.

MLOps

MLOps, makine öğrenmesi modellerinin geliştirilmesini, canlıya alınmasını ve canlıda izlenmesini sistemli hâle getiren pratikler bütünüdür. Yazılım geliştirme ile işletimin birleşimini ifade eden DevOps kavramının makine öğrenmesine uyarlanmış hâlidir. Temel dersi şudur: bir model bir kez yapılıp bırakılmaz; dünya değiştikçe model eskir. Bu nedenle modelin bütün yaşam döngüsü kayıt altında, kontrollü ve geri alınabilir olmalıdır.

Model sürümleme

Her modelin hangi veriyle, hangi kodla ve hangi ayarlarla üretildiği kaydedilmelidir. Bu kayıt, hem bir modelin neden öyle davrandığını anlamayı hem de gerektiğinde önceki bir sürüme dönmeyi mümkün kılar. MLflow ve DVC (*Data Version Control*) gibi araçlar bu amaçla kullanılır.

Örneğin yeni eğitilen bir fiyatlandırma modeli canlıya alındıktan sonra satışlar düşmeye başlarsa, ekip birkaç dakika içinde bir önceki sürüme dönebilmelidir. Sürümleme yoksa, önceki modelin hangi veriyle eğitildiği bilinmez ve aynı modeli yeniden üretmek mümkün olmaz. Yazılımda kod sürümlenir; makine öğrenmesinde kodla birlikte veri ve model de sürümlenmelidir, çünkü aynı kod farklı veriyle farklı bir model üretir.

Deney takibi

Model geliştirme, çok sayıda denemeden oluşur. Her denemenin ayarları ve sonucu kaydedilmezse, neyin neden işe yaradığı unutulur, başarılı bir sonuç yeniden üretilemez ve başarısız denemeler tekrar tekrar yapılır.

Örneğin bir ekip üç ay boyunca yüzlerce deneme yaptıktan sonra, en iyi sonucu veren modelin hangi öğrenme hızıyla ve hangi veri sürümüyle eğitildiğini hatırlayamayabilir. Deney takibi, her denemenin ayarlarını, kullanılan veriyi ve elde edilen metrikleri otomatik olarak kaydeder ve karşılaştırmayı mümkün kılar.

Konteyner ve orkestrasyon ile dağıtım

Model her ortamda aynı biçimde çalışmalıdır. Geliştiricinin bilgisayarında çalışan ama sunucuda farklı bir kütüphane sürümü yüzünden bozulan bir model, sık karşılaşılan bir sorundur. **Konteyner** (*container*), uygulamayı bütün bağımlılıklarıyla birlikte paketleyen ve her ortamda aynı biçimde çalıştıran yapıdır; Docker bu alanda en yaygın araçtır. Kubernetes ise çok sayıda konteynerin çalıştırılmasını, ölçeklenmesini ve arızalandığında yeniden başlatılmasını yönetir.

Örneğin bir model, bir kampanya günü normalin on katı istek aldığında, Kubernetes modeli çalıştıran konteyner sayısını otomatik olarak artırabilir ve yük azaldığında yeniden düşürebilir. Ders şudur: "benim bilgisayarımda çalışıyordu" bir mazeret değildir; çalışma ortamı da kodun bir parçası olarak tanımlanmalıdır.

Makine öğrenmesi için sürekli entegrasyon ve dağıtım

Sürekli entegrasyon ve sürekli dağıtım (*continuous integration / continuous delivery*, CI/CD), bir değişikliğin otomatik olarak test edilip onaylandıktan sonra canlıya alınmasını sağlayan süreçtir. Makine öğrenmesinde bu süreç kod testlerinin yanında veri ve model testlerini de içerir.

Örneğin yeni bir model, canlıya alınmadan önce otomatik olarak şu kontrollerden geçirilebilir: eğitim verisinde beklenmedik boş değerler var mı, yeni model test verisinde eskisinden daha kötü mü, yanıt süresi kabul edilebilir sınırın içinde mi. Kontrollerden biri başarısız olursa model canlıya alınmaz. Ders şudur: insan eliyle yapılan her adım bir hata ihtimalidir; tekrar eden kontroller otomatikleştirilmelidir.

Canlıdaki modeli izleme

Bir model canlıya alındığı anda eskimeye başlar. Gerçek dünyadaki veri, modelin eğitildiği veriden zamanla uzaklaşır. Girdi verisinin dağılımının zaman içinde değişmesine **veri kayması** (*data drift*) denir. Girdi ile sonuç arasındaki ilişkinin kendisinin değişmesine ise kavram kayması (*concept drift*) denir.

Örneğin bir dolandırıcılık tespit modeli, dolandırıcıların geçen yıl kullandığı yöntemleri öğrenmiştir. Dolandırıcılar yöntem değiştirdiğinde model yeni yöntemleri tanımaz; teknik olarak hatasız çalışmaya devam eder ama giderek daha az dolandırıcılık yakalar. İzleme yoksa bu düşüş ancak büyük kayıplar ortaya çıktığında fark edilir. Bu nedenle canlıdaki modelin performansı ve girdi verisinin dağılımı sürekli izlenmeli, belirli bir eşiğin altına düşüldüğünde model yeniden eğitilmelidir.

Özellik deposu

Modelin kullandığı girdilere **özellik** (*feature*) denir: bir müşterinin son 30 gündeki harcaması, son siparişinden bu yana geçen gün sayısı gibi. **Özellik deposu** (*feature store*), bu özelliklerin tek bir yerde, tek bir tanımla hesaplanıp saklandığı sistemdir.

Özellik deposunun çözdüğü temel sorun, eğitim ile canlı kullanım arasındaki tutarsızlıktır (*training-serving skew*). Örneğin "son 30 günlük harcama" özelliği eğitimde iadeler düşülerek, canlı sistemde ise iadeler düşülmeden hesaplanırsa, model eğitimde gördüğünden farklı bir girdiyle çalışır ve sessizce yanlış sonuç üretir. Bu hata herhangi bir uyarı vermez; yalnızca performans beklenmedik biçimde düşer. Ayrıca aynı özelliğin farklı ekipler tarafından tekrar tekrar hesaplanması da önlenmiş olur.

Hazır platformlar

Kubeflow, Amazon SageMaker ve Google Vertex AI gibi platformlar; sürümleme, deney takibi, dağıtım ve izleme işlevlerinin önemli bir kısmını hazır olarak sunar. Bu platformlar, bütün altyapıyı sıfırdan kurma yükünü ortadan kaldırır.

Ancak hazır platformların da bir bedeli vardır. Bir platforma derinlemesine bağlanan bir sistem, daha sonra başka bir platforma taşınmak istendiğinde büyük bir yeniden yazım gerektirebilir (*vendor lock-in*). Ayrıca kullanım arttıkça maliyet de artar. Ders şudur: hazır platform zaman kazandırır, ancak bağımlılık ve uzun vadeli maliyet baştan hesaba katılmalıdır.

Genel Ders

Model zamanla eskir. Bu nedenle her şeyin kaydı tutulmalı, her değişiklik otomatik olarak test edilmeli, canlıdaki model sürekli izlenmeli ve bir sorun çıktığında hızla bir önceki çalışan sürüme dönülebilmelidir.

Analitik Mühendisliği

Analitik mühendisliği, veri mühendisliği ile veri analizi arasında yer alan bir roldür. Temel dersi şudur: ham veri tek başına kimsenin işine yaramaz; değer, veriyi bir kurumdaki herkesin güvenip kullanabileceği temiz, anlaşılır ve tutarlı bir hâle getirmekte ortaya çıkar.

İleri seviye SQL

SQL (*Structured Query Language*), ilişkisel veritabanlarında veriyi sorgulamak için kullanılan dildir ve verinin ortak dilidir. Bir soruyu doğru bir sorguya çevirebilmek, cevabın doğru olmasının ilk koşuludur.

SQL'in en tehlikeli hataları sessiz olanlardır. Örneğin bir sipariş tablosu, her siparişin birden fazla satırı bulunan bir ödeme tablosuyla birleştirildiğinde (*JOIN*), her sipariş ödeme sayısı kadar tekrarlanır. Bu durumda toplam ciro hesaplanırsa, sonuç herhangi bir hata mesajı vermeden olduğundan iki ya da üç kat büyük çıkar. Sorgu çalışmıştır, rakam makul görünür, ama yanlıştır. Bu nedenle bir birleştirmeden sonra satır sayısının beklendiği gibi olup olmadığı kontrol edilmelidir.

dbt ile veri modelleme

dbt (*data build tool*), veri dönüşümlerinin SQL ile, ancak yazılım geliştirme disipliniyle yazılmasını sağlayan bir araçtır. Her dönüşüm adımı ayrı bir dosyada tanımlanır, adımlar arasındaki bağımlılıklar otomatik olarak izlenir ve bütün süreç tek bir komutla yeniden çalıştırılabilir.

dbt'nin öğrettiği ders, veri dönüşümlerinin de kod gibi ele alınması gerektiğidir. Örneğin bir analistin kendi bilgisayarında elle çalıştırdığı ve bir sonraki adımı da yine elle tetiklediği yirmi ayrı sorgu, analist izne çıktığında ya da kurumdan ayrıldığında çalışmaz hâle gelir. Aynı dönüşümler düzenli, belgelenmiş ve otomatik çalıştırılabilir adımlar hâline getirildiğinde, herkes tarafından anlaşılabilir ve bakımı yapılabilir olur.

Boyutsal modelleme

Boyutsal modelleme (*dimensional modeling*), veriyi insanların soru sorma biçimine göre düzenleme yaklaşımıdır (Kimball ve Ross, 2013). Bu yaklaşımda iki tür tablo bulunur. Olgu tablosu (*fact table*), ölçülebilir olayları tutar: satış, tıklama, sipariş. Boyut tabloları (*dimension tables*) ise bu olayları tarif eden bilgileri tutar: müşteri, ürün, tarih.

Bir çevrimiçi mağazada her satış bir olaydır ve olgu tablosunda yer alır:

satis_id	tarih_id	musteri_id	urun_id	adet	tutar
1	20261001	501	12	2	400
2	20261001	502	7	1	1500
3	20261002	501	7	1	1500

Bu tabloda yalnızca kimlik numaraları ve ölçülebilir değerler vardır. Müşterinin, ürünün ve tarihin ayrıntıları ayrı boyut tablolarında tutulur:

musteri_id	ad	sehir
501	Ayşe	Bursa
502	Mehmet	İzmir

urun_id	urun_adi	kategori
7	Kulaklık	Elektronik
12	Kupa	Mutfak

tarih_id	gun	ay	yil
20261001	1	Ekim	2026
20261002	2	Ekim	2026

Olgu tablosu ortada, boyut tabloları onun çevresinde yer aldığı için bu yapıya yıldız şema (*star schema*) denir. Bu yapıda "Ekim ayında Bursa'da Elektronik kategorisinden ne kadar satış yapıldı?" sorusu kolayca yanıtlanır:

```
SELECT SUM(s.tutar)
FROM satislar s
JOIN musteriler m ON s.musteri_id = m.musteri_id
JOIN urunler u    ON s.urun_id = u.urun_id
JOIN tarihler t   ON s.tarih_id = t.tarih_id
WHERE m.sehir = 'Bursa'
  AND u.kategori = 'Elektronik'
  AND t.ay = 'Ekim';
```

Sonuç 1500'dür. Şehir, kategori ve ay değiştirilerek aynı yapıyla çok sayıda farklı soru sorulabilir. Boyutsal modellemenin dersi şudur: veri, depolama kolaylığına göre değil, insanların soracağı sorulara göre düzenlenmelidir.

Veri testleri

Bir rakama güvenilmiyorsa, o rakamın değeri yoktur. Veri testleri, verinin belirli kurallara uyduğunu her gün otomatik olarak kontrol eder ve bozuk verinin raporlara ulaşmasını önler. Yukarıdaki tablolar için yazılabilecek testler şunlardır:

- `satis_id` tekrar etmemelidir. Aynı satış iki kez kaydedilirse ciro iki kat görünür.
- `tutar` negatif olmamalıdır. Eksi 400 TL'lik bir satış, bir yerde hata olduğunu gösterir.
- `musteri_id` boş olmamalıdır. Sahibi olmayan bir satış, müşteri bazlı raporlarda kaybolur.

- Olgu tablosundaki her `musteri_id`, müşteri tablosunda bulunmalıdır. Satış tablosunda 503 numaralı bir müşteri bulunup müşteri tablosunda bulunmuyorsa, bu satış şehir bazlı raporlarda görünmez.
- Bir önceki günün satış sayısı sıfır olmamalıdır. Sıfırsa, büyük olasılıkla veri hiç gelmemiştir.

Bu testlerden biri başarısız olduğunda, rapor yayımlanmadan önce uyarı verilir.

Dokümantasyon

Dokümantasyon, bir verinin ne anlama geldiğini tanımlar. Aynı kelimenin farklı ekipler tarafından farklı anlamlarda kullanılması, bir kurumdaki en yaygın veri sorunlarından biridir.

Örneğin bir kurumda üç ekip "aktif kullanıcı" kavramını kullanmaktadır. Pazarlama ekibi için aktif kullanıcı, son 30 günde siteye giren herkestir. Ürün ekibi için son 7 günde en az bir işlem yapan herkestir. Finans ekibi için ise son 30 günde ödeme yapan herkestir. Yönetim toplantısında üç farklı rakam ortaya çıkar: 50.000, 12.000 ve 3.000. Her ekip kendi rakamının doğru olduğunu söyler. Üç rakam da doğrudur, ancak farklı şeyleri ölçmektedir. Dokümantasyon bu sorunu her kavrama ayrı ve açık bir tanım vererek çözer:

- `aktif_kullanici`: Son 30 günde en az bir kez giriş yapan ve test hesabı olmayan kullanıcı.
- `islem_yapan_kullanici`: Son 7 günde en az bir işlem yapan kullanıcı.
- `odeyen_kullanici`: Son 30 günde en az bir ödeme yapan kullanıcı.

Git ile çalışma

Git, dosyalardaki değişiklikleri kaydeden bir sürüm kontrol sistemidir. Veri tanımları ve dönüşümleri de zaman içinde değişir; kimin neyi, ne zaman ve neden değiştirdiğinin bilinmesi gerekir.

Örneğin bir raporda aylık ciro bir gün aniden %15 düşmüş görünüyorsa, bunun gerçek bir düşüş mü yoksa birinin ciro tanımından iadeleri çıkarması mı olduğu anlaşılmalıdır. Git kaydı, ciro tanımının o gün değiştirildiğini, kimin değiştirdiğini ve gerekçesini gösterir. Ayrıca değişiklik hatalıysa eski tanıma dönmeyi mümkün kılar.

ELT mantığı

Geleneksel yaklaşımda veri kaynaktan alınır, dönüştürülür ve sonra depoya yüklenir (*extract, transform, load*, ETL). Modern yaklaşımda ise veri önce olduğu gibi depoya yüklenir, dönüştürme sonra depo içinde yapılır (*extract, load, transform*, ELT).

ELT'nin temel avantajı, ham verinin her zaman elde kalmasıdır. Örneğin bir dönüşüm adımında bir hata yapıldığı aylar sonra fark edilirse, ETL yaklaşımında ham veri artık yoktur ve hatalı veri düzeltilemez. ELT yaklaşımında ise ham veri saklanmıştır; dönüşüm düzeltilip bütün geçmiş veri baştan hesaplanabilir. Aynı şekilde ilk başta öngörülmemiş yeni bir soru ortaya çıktığında, ham veriden yeni bir dönüşüm türetilebilir.

Genel Ders

Bir kurumda herkes aynı rakama bakmalı ve o rakama güvenebilmelidir. Bu güven; doğru yazılmış sorgularla, insanların sorularına göre düzenlenmiş tablolarla, otomatik testlerle, açık tanımlarla, değişikliklerin kaydıyla ve ham verinin korunmasıyla kurulur.

İş Zekâsı

İş zekâsının temel dersi şudur: veri, karar veren kişi onu bir bakışta anlayabildiğinde değer kazanır. Doğru rakam, doğru kişiye, anlaşılır biçimde ve zamanında ulaşmalıdır.

Araçlar

Power BI, Tableau ve Looker gibi araçlar aynı temel işi yapar: veriyi kaynağa bağlar, düzenler ve görsel olarak sunar. Arayüzleri ve bazı yetenekleri farklıdır, ancak mantıkları ortaktır. Bu nedenle birini iyi öğrenen biri diğerlerine kolayca geçer. Kalıcı beceri araç değil; doğru veri modelini kurmak, doğru hesaplamayı yapmak ve doğru şeyi göstermektir.

Gösterge paneli tasarımı

Gösterge paneli (*dashboard*), önemli metriklerin tek bir ekranda toplandığı görünümdür. İyi bir gösterge panelinin ilkesi, az ama doğru bilgi göstermektir. Bir ekrana otuz grafik konduğunda kullanıcı hiçbirine dikkatle bakmaz.

Örneğin bir satış müdürü sabah gösterge panelini açtığında, "dün iyi bir gün müydü?" sorusunun cevabını beş saniye içinde görebilmelidir. Bu nedenle en önemli rakam, gözün ilk baktığı sol üst köşede ve büyük punto ile yer almalı; hedefle ya da geçen dönemle karşılaştırması yanında verilmeli; ayrıntılar aşağıda ve ikinci planda kalmalıdır.

Hesaplama dili

DAX (*Data Analysis Expressions*), Power BI'da hesaplamalar yapmak için kullanılan formül dilidir. DAX'in ve genel olarak BI hesaplamalarının öğrettiği ders, bir hesaplamanın nerede ve nasıl yapıldığının sonucu değiştirdiğidir.

Örneğin "ortalama sipariş tutarı" iki farklı biçimde hesaplanabilir. Birinci yöntemde her günün ortalama sipariş tutarı hesaplanır ve bu günlük ortalamaların ortalaması alınır. İkinci yöntemde ise toplam ciro, toplam sipariş sayısına bölünür. Sipariş sayısı günden güne çok farklıysa bu iki yöntem farklı sonuç verir: az siparişin geldiği bir günün ortalaması, birinci yöntemde çok siparişin geldiği bir günle aynı ağırlığı taşır. Hangi yöntemin doğru olduğu, sorulan soruya bağlıdır ve bu bilinçli olarak seçilmelidir.

Veri modeli ve tablolar arası ilişkiler

BI araçlarında tablolar arasındaki ilişkiler, hesaplamaların nasıl yapılacağını belirler. Görsel olarak kusursuz bir grafik, yanlış kurulmuş bir veri modelinin üzerinde yanlış bilgi gösterir.

Örneğin satış tablosu ile müşteri tablosu arasındaki ilişki yanlış kurulursa ve müşteri tablosunda bir müşteri iki kez yer alıyorsa, o müşterinin bütün satışları iki kez sayılır. Grafik düzgün görünür, rakamlar makul aralıkta kalır; yalnızca

sonuç yanlıştır. Bu tür hatalar, analitik mühendisliğindeki veri testleriyle ve modelin dikkatle kurulmasıyla önlenir.

Temel performans göstergelerini belirlemek

Temel performans göstergesi (*key performance indicator*, KPI), bir kurumun hedeflerine ne kadar yaklaştığını gösteren seçilmiş metriktir. Her şeyi ölçmek, hiçbir şeyi ölçmemekle eşdeğerdir; kurumun gerçekten neyi önemsediğine karar verilip ona odaklanılmalıdır.

Örneğin bir abonelik şirketi için "yeni kayıt sayısı" olumlu bir gösterge gibi görünür. Ancak bir ayda 1000 yeni kayıt ve 1200 iptal varsa, şirket büyümüyor, küçülüyordur. Bu şirket için asıl önemli gösterge, iptal oranı ve net abone değişimi olabilir. Yanlış seçilmiş bir KPI, kurumun yanlış yöne doğru başarılı biçimde ilerlemesine neden olur.

Rapor otomasyonu ve paylaşım

Elle hazırlanan raporlar hem zaman kaybıdır hem de hataya açıktır. Rapor kendiliğinden güncellenmeli ve doğru kişiye doğru zamanda ulaşmalıdır.

Örneğin her pazartesi bir çalışan, farklı sistemlerden rakamları Excel'e kopyalayıp yönetim sunumunu hazırlıyorsa, bir gün yanlış hücreyi kopyalayacaktır ve bu hata fark edilmeden kararlara yansıyabilir. Otomatik rapor bu riski ortadan kaldırır. Paylaşımda ayrıca erişim kontrolü gözetilmelidir: bir bölge müdürü yalnızca kendi bölgesinin verisini görmeli, çalışanların maaş bilgisi yalnızca yetkili kişilere açık olmalıdır. Bu tür satır düzeyinde güvenlik (*row-level security*), BI araçlarının temel özelliklerinden biridir.

Genel Ders

İş zekâsında amaç veriyi göstermek değil, karar verdirmektir. Bunun için doğru göstergeler seçilmeli, hesaplamalar bilinçli kurulmalı, veri modeli doğrulanmalı, ekran sade tutulmalı ve raporlar otomatik ve yetkiye göre paylaşılmalıdır.

Büyük Veri

Büyük verinin temel dersi şudur: veri tek bir bilgisayara sığmayacak kadar büyüdüğünde, iş parçalara bölünüp birçok bilgisayara dağıtılmak zorundadır. Bu noktadan sonra asıl zorluk hesaplamanın kendisi değil, veriyi taşımak, bölmek ve parçaları eşgüdüm içinde yönetmektir.

Dağıtık sistemlerin temel mantığı

Dağıtık sistem (*distributed system*), birden fazla bilgisayarın ağ üzerinden birlikte çalışarak tek bir işi yaptığı sistemdir. Bir bilgisayarın yetmediği yerde iş yüz bilgisayara bölünür. Ancak yüz bilgisayar bulunduğunda bazıları arızalanır, bazıları yavaş kalır ve aralarındaki iletişim zaman alır.

Örneğin her bilgisayarın bir yılda bir kez arızalandığı varsayılsın. Tek bir bilgisayarla arıza yılda bir kez yaşanır. Bin bilgisayarlı bir kümede ise neredeyse her gün birkaç arıza yaşanır. Bu nedenle dağıtık sistemler, arızayı bir istisna olarak değil olağan bir durum olarak ele alacak biçimde tasarlanır: arızalanan bir makinenin işi otomatik olarak başka bir makineye verilir ve iş kaldığı yerden devam eder.

Hadoop ve HDFS

Hadoop, büyük verinin çok sayıda sıradan bilgisayarda saklanması ve işlenmesi için geliştirilmiş açık kaynaklı bir ekosistemdir. Bunun depolama katmanı olan HDFS (*Hadoop Distributed File System*), büyük dosyaları bloklara böler ve her bloğun birkaç kopyasını farklı makinelerde saklar (Shvachko vd., 2010).

Örneğin 1 TB büyüklüğündeki bir kayıt dosyası, varsayılan ayarlarda 128 MB'lık bloklara bölünür ve her blok üç farklı makinede saklanır. Bir makine tamamen çökse bile verinin diğer iki kopyası erişilebilir kalır. Bu yaklaşımın dersi, güvenilirliğin pahalı ve hiç bozulmayan donanımla değil, ucuz donanımın çoğaltılmasıyla sağlanabileceğidir.

Apache Spark

Apache Spark, büyük veriyi çok sayıda makinede paralel olarak işleyen bir motordur (Zaharia vd., 2012). Hadoop'un ilk işleme modeli olan MapReduce, her adımın sonucunu diske yazıp bir sonraki adımda yeniden okuyordu. Spark ise ara sonuçları mümkün olduğunca bellekte tutar.

Örneğin aynı veri üzerinde art arda on adım çalıştıran bir analizde, MapReduce on kez diske yazıp okurken, Spark veriyi bellekte tutarak aynı işi çok daha hızlı yapar. Spark'ın ardındaki iki temel ilke şunlardır: disk erişimi bellek erişiminden çok daha yavaş olduğu için mümkün olduğunca belleğe yazılmalıdır; veriyi ağ üzerinden taşımak pahalı olduğu için hesaplama verinin bulunduğu makineye götürülmelidir.

Kafka ile veri akışı

Bazı veriler bir kez gelip bitmez, sürekli akar: kart işlemleri, sensör ölçümleri, kullanıcı tıklamaları. Apache Kafka, bu sürekli akışı kaybetmeden, sırasını bozmadan ve birçok tüketiciye aynı anda sunacak biçimde taşıyan bir sistemdir (Kreps, Narkhede ve Rao, 2011).

Örneğin bir bankada her saniye binlerce kart işlemi gerçekleşir. Bu işlemler Kafka'ya yazılır. Dolandırıcılık tespit sistemi, muhasebe sistemi ve müşteriye bildirim gönderen sistem, aynı akışı birbirinden bağımsız olarak okur. Sistemlerden biri bir süre çalışmazsa, yeniden çalıştığında kaldığı yerden okumaya devam eder; hiçbir işlem kaybolmaz.

Toplu ve akış işleme

Verinin işlenmesinde iki temel yaklaşım vardır. Toplu işleme (*batch processing*), belirli bir süre boyunca biriken verinin bir seferde işlenmesidir. Akış işleme (*stream processing*), verinin geldiği anda işlenmesidir.

Bazı sorular bekleyebilir, bazıları bekleyemez. "Geçen ay ne kadar satış yapıldı?" sorusu gece toplu olarak hesaplanabilir. "Bu kart işlemi şüpheli mi?" sorusunun cevabı ise birkaç saniye içinde verilmelidir; aksi hâlde dolandırıcılık tamamlanmış olur. Akış işleme daha karmaşık ve daha pahalıdır. Bu nedenle gerçekten anlık cevap gerekmedikçe toplu işleme tercih edilir. Ders şudur: "gerçek zamanlı" bir değer değil, bir maliyettir; yalnızca karşılığı olduğunda ödenmelidir.

Veri formatları

Verinin diskte nasıl saklandığı, ne kadar hızlı okunabileceğini belirler. CSV gibi metin formatları veriyi satır satır ve sıkıştırmadan saklar. Parquet, veriyi sütun sütun ve sıkıştırılmış biçimde saklayan bir formattır. Avro ise veriyi satır satır ama ikili (*binary*) ve şemasıyla birlikte saklar.

Örneğin 100 sütunlu bir tablodan yalnızca iki sütun gerekiyorsa, CSV dosyasında bütün satırların bütün sütunları okunmak zorundadır. Parquet ise yalnızca gereken iki sütunu okur ve aynı iş onlarca kat hızlanır. Aynı sütundaki değerler birbirine benzediği için sütun bazlı saklama sıkıştırmayı da daha verimli kılar. Avro ise verinin sürekli ve satır satır yazıldığı akışlarda daha uygundur. Ders şudur: veri formatı, verinin en çok nasıl okunacağına ya da yazılacağına göre seçilir.

Bölümleme ve performans ayarı

Bölümleme (*partitioning*), verinin belirli bir sütuna göre ayrı parçalara bölünerek saklanmasıdır. Veri, en çok nasıl sorgulanacaksa ona göre bölünmelidir.

Örneğin satış verisi tarihe göre ayrı klasörlere bölünürse, "dünkü satışlar" sorgusu yalnızca bir günlük klasörü okur ve yıllarca biriken verinin taranmasına gerek kalmaz. Ancak yanlış bölümleme ters etki yapar. Veri gereğinden ince

bölünürse, örneğin her müşteri için ayrı bir klasör açılırsa, milyonlarca küçük dosya oluşur ve sistem bu dosyaları yönetmekle uğraşırken yavaşlar.

Bölümlemede ikinci bir sorun dengesizliktir (*data skew*). Örneğin veri ülkeye göre bölünmüşse ve satışların %90'ı tek bir ülkeden geliyorsa, işin %90'ı tek bir makineye düşer. Diğer makineler işlerini bitirip beklerken bütün iş o tek makinenin bitirmesini bekler. Dağıtık sistemin hızı, en yavaş parçasının hızıyla sınırlıdır.

Genel Ders

Büyük veride asıl maliyet hesaplama değil, veriyi taşımak ve okumaktır. Veri az taşınmalı, hesaplama verinin yanına götürülmeli, yalnızca gereken kısım okunmalı ve veri, sorgulanacağı biçime göre dengeli parçalara bölünmelidir. Arıza olağan kabul edilmeli ve sistem buna göre tasarlanmalıdır.

Bulut Veri Platformları

Bulut veri platformları, sunucu kurmayı, disk satın almayı ve makine yönetmeyi gereksiz kılar. Ancak bunun karşılığında her sorgu, her saniyelik işlem süresi ve her gigabaytlık depolama ölçülür ve faturalandırılır. Temel ders şudur: bulut neredeyse sınırsız güç sağlar; asıl beceri bu gücü kontrollü ve bilinçli kullanmaktır.

Snowflake

Snowflake, depolama ile hesaplamayı birbirinden ayıran bir bulut veri ambarıdır (Dageville vd., 2016). Veri tek bir yerde durur; onu işleyen hesaplama kaynakları ise gerektiğinde açılıp kapatılabilir ve birbirinden bağımsız olarak büyütülebilir.

Örneğin finans ekibi ay sonunda ağır raporlar çalıştırırken, pazarlama ekibi aynı veriye kendi ayrı hesaplama kaynağıyla erişir; iki ekip birbirini yavaşlatmaz. İş bittiğinde hesaplama kaynakları kapanır ve ücret işlemez. Snowflake ayrıca veriyi kopyalamadan başka bir kuruma erişim verilmesine olanak tanır. Böylece verinin tek ve güncel bir kopyası kalır; kopyalar arasında tutarsızlık oluşmaz.

Databricks

Databricks, veri gölü ile veri ambarını birleştiren **lakehouse** yaklaşımını temel alan bir platformdur (Armbrust vd., 2021). Delta Lake, ucuz nesne depolamasında duran dosyalara bir veritabanının güvenilirliğini kazandıran açık kaynaklı bir katmandır (Armbrust vd., 2020).

Geleneksel yaklaşımda ham veri ucuz bir depoda tutulur, analiz için ayrıca bir veri ambarına kopyalanırdı; bu iki kopya, iki maliyet ve zamanla birbirinden ayrışan iki farklı "gerçek" anlamına geliyordu. Delta Lake sayesinde aynı dosyalar üzerinde hem veri bilimi hem raporlama yapılabilir; ayrıca "bu tablonun dün saat üçteki hâli neydi?" sorusu yanıtlanabilir (*time travel*). Unity Catalog ise kimin hangi veriye erişebildiğini tek bir yerden yönetir.

BigQuery

BigQuery, Google'ın sunucusuz (*serverless*) veri ambarıdır. Kullanıcı hiçbir sunucu ya da küme yönetmez; yalnızca sorgu yazar. Talebe göre ücretlendirme modelinde ödenen ücret, sorgunun taradığı veri miktarına bağlıdır.

Bu model, iyi SQL yazmayı doğrudan bir maliyet becerisine dönüştürür. Örneğin 10 TB'lık bir tabloda bütün sütunları getiren bir sorgu, bütün tabloyu tarar ve tek bir sorgu için yüksek bir ücret doğurur. Yalnızca gereken iki sütunun seçilmesi ve tarihe göre bölümlenmiş tabloda tarih filtresi kullanılması, aynı cevabı çok daha düşük maliyetle verir.

Redshift ve AWS veri servisleri

Amazon Web Services (AWS), tek bir büyük sistem yerine her biri belirli bir işi yapan birçok servis sunar. S3 nesne depolaması sağlar; Glue veriyi taşır ve dönüştürür; Athena, S3'teki dosyaları doğrudan SQL ile sorgular; Redshift ise veri ambarı görevi görür.

Örneğin bir uygulamanın günlük kayıt dosyaları S3'e yazılır, Glue bu dosyaları temizleyip düzenler, Athena ile hızlı keşif sorguları yapılır ve düzenli raporlar için veri Redshift'e yüklenir. Bu yaklaşım esnektir: her parça ihtiyaca göre seçilebilir ve değiştirilebilir. Bedeli ise parçaları birbirine bağlamanın, aralarındaki yetkileri ayarlamanın ve bütünü izlemenin kullanıcıya kalmasıdır.

Yetkilendirme ve güvenlik

Bulutta veri, yanlış bir ayarla bütün internete açık hâle gelebilir. Herkese açık bırakılmış depolama alanları nedeniyle çok sayıda kurumun müşteri verisi, hiçbir saldırı yapılmadan yalnızca yanlış bir erişim ayarı yüzünden ortaya çıkmıştır.

Buradaki temel ilke en az yetki ilkesidir (*principle of least privilege*): her kullanıcıya ve her sisteme yalnızca işini yapmaya yetecek kadar yetki verilmelidir (Saltzer ve Schroeder, 1975). Örneğin raporlama yapan bir servis, veriyi yalnızca okuyabilmeli; silme ya da değiştirme yetkisine sahip olmamalıdır. Bu servisin erişim anahtarı bir şekilde ele geçirilirse, verilecek zarar okumayla sınırlı kalır.

Sertifika sınavları

Bulut sağlayıcılarının sertifikaları, bir platformun bilindiğini göstermenin hızlı ve tanınan bir yoludur. Ancak sertifikanın asıl değeri, platformun bütün parçalarının sistemli biçimde öğrenilmesini zorunlu kılmasıdır.

Örneğin günlük işinde yalnızca üç servis kullanan bir mühendis, sınava hazırlanırken başka on servisin ne işe yaradığını öğrenir. Bir sonraki tasarımda, daha önce bilmediği için elle yazdığı bir işlevin zaten hazır bir serviste bulunduğunu fark eder ve daha uygun bir çözüm seçer.

Genel Ders

Bulutta her şey ölçülür ve faturalandırılır. Hesaplama gücü ihtiyaç kadar açılmalı ve iş bitince kapatılmalı; yalnızca gereken veri okunmalı; hazır servislerin esnekliği ile bunları birbirine bağlamanın yükü dengelenmeli; erişim en az yetki ilkesine göre sıkı tutulmalıdır.

Veritabanı Yönetimi

Veritabanı, bir kurumun hafızasıdır. Veritabanı yönetiminin temel dersi şudur: hızlı çalışmak önemlidir, ancak asıl önemli olan verinin hiçbir zaman kaybolmaması, hiçbir zaman bozulmaması ve her zaman erişilebilir olmasıdır. Hız bu üç koşuldan sonra gelir.

Kurulum ve yapılandırma

Veritabanlarının varsayılan ayarları, mümkün olan en geniş ortam yelpazesinde çalışabilmek için temkinli seçilmiştir; dolayısıyla belirli bir ortama nadiren tam uyar.

Örneğin PostgreSQL'in veri önbelleği için ayırdığı bellek miktarını belirleyen `shared_buffers` ayarı, varsayılan olarak 128 MB'tır (PostgreSQL Global Development Group, 2024). 64 GB belleği olan bir sunucuda bu ayar değiştirilmezse, makinenin belleğinin büyük kısmı veritabanı tarafından doğrudan kullanılmaz ve performans gereksiz yere düşük kalır. Ders şudur: varsayılan ayarlar başlangıç noktasıdır, son nokta değil.

Index'ler ve sorgu optimizasyonu

Index, bir tablodaki satırlara belirli bir sütun üzerinden hızlı erişim sağlayan yardımcı veri yapısıdır. Bir kitabın sonundaki dizin gibi çalışır: aranan konu, bütün kitap okunmadan bulunur.

Örneğin 10 milyon satırlık bir müşteri tablosunda e-posta adresine göre arama yapılıyorsa ve e-posta sütununda index yoksa, veritabanı 10 milyon satırı tek tek okur. Index varsa, aranan satır birkaç adımda bulunur. Ancak her index'in bir bedeli vardır: tabloya her yeni kayıt eklendiğinde ya da bir kayıt değiştirildiğinde bütün index'lerin de güncellenmesi gerekir. Her sütuna index eklenirse okuma hızlanır ama yazma işlemleri yavaşlar ve disk kullanımı artar. Ders şudur: index, en sık yapılan sorgulara göre seçilmelidir.

Yedekleme ve geri yükleme

Bir yedeğin değeri, ondan geri dönülebildiği ölçüdedir. Geri yüklenmesi hiç denenmemiş bir yedeğin çalışacağı varsayılamaz.

Örneğin bir kurum yıllarca her gece yedek aldığını düşünür. Gerçek bir veri kaybı yaşandığında yedek dosyalarının bozuk olduğu, bazı tabloların yedeğe hiç dahil edilmediği ya da geri yüklemenin üç gün sürdüğü ortaya çıkar. Bu nedenle yedekten geri dönüş düzenli aralıklarla, gerçek koşullara yakın bir ortamda test edilmeli ve geri yüklemenin ne kadar sürdüğü ölçülmelidir.

Replikasyon ve yüksek erişilebilirlik

Replikasyon (*replication*), verinin bir veritabanı sunucusundan bir ya da daha fazla sunucuya sürekli olarak kopyalanmasıdır. Yüksek erişilebilirlik (*high avail-*

ability), ana sunucu çöktüğünde kopyalardan birinin görevi otomatik olarak devralmasıyla sağlanır.

Örneğin ana veritabanı sunucusu bir donanım arızası nedeniyle çöktüğünde, yedek sunucu birkaç saniye içinde ana sunucu rolünü üstlenir ve kullanıcılar bir kesinti fark etmez. Ancak replikasyon bir yedek değildir. Birisi yanlışlıkla bir tabloyu silerse, bu silme işlemi de anında bütün kopyalara aktarılır ve veri her yerde kaybolur. Replikasyon donanım arızasına karşı korur, insan hatasına karşı korumaz; insan hatasına karşı yedek gerekir.

Kullanıcı ve yetki yönetimi

Her kullanıcı ve her uygulama yalnızca ihtiyacı olan işlemleri yapabilmelidir. Bulut güvenliğindeki en az yetki ilkesi veritabanı için de geçerlidir.

Örneğin rapor hazırlayan bir analistin yalnızca okuma yetkisine sahip olması gerekir. Silme yetkisi varsa, bir gün yanlış bir komutla bütün tabloyu silebilir. Aynı şekilde bir uygulamanın veritabanına bağlanırken kullandığı hesap, hiçbir zaman yönetici yetkisiyle çalışmamalıdır; uygulamada bir güvenlik açığı bulunursa, saldırgan bu yetkiyle bütün veritabanını ele geçirebilir.

Transaction, kilitleme ve izolasyon

Transaction, ya tamamı gerçekleşen ya da hiçbiri gerçekleşmeyen işlemler bütünüdür. Veritabanları, transaction'ların güvenilirliğini dört özellikle tanımlar: bölünmezlik (*atomicity*), tutarlılık (*consistency*), yalıtım (*isolation*) ve kalıcılık (*durability*); kısaca ACID (Härder ve Reuter, 1983).

Örneğin bir para transferinde bir hesaptan 100 TL düşülür ve başka bir hesaba 100 TL eklenir. Bu iki adımın arasında sistem çökerse, para bir hesaptan çıkmış ama diğerine ulaşmamış olur. Transaction, bu iki adımı birlikte yapar ya da hiçbirini yapmaz; para kaybolmaz.

Kilitleme (*locking*), iki işlemin aynı anda aynı satırı değiştirip birbirinin sonucunu ezmesini önler. Örneğin iki kişi aynı anda son kalan uçak koltuğunu almaya çalıştığında, kilitleme yalnızca birinin başarılı olmasını sağlar.

İzolasyon seviyesi ise "bir işlem, henüz tamamlanmamış başka bir işlemin etkilerini görebilir mi?" sorusunun cevabıdır. Daha sıkı izolasyon daha güvenlidir ama işlemlerin birbirini daha çok beklemesine yol açtığı için daha yavaştır; daha gevşek izolasyon daha hızlıdır ama belirli tutarsızlıklara izin verir (Berenson vd., 1995). Ders şudur: güvenlik ile hız arasındaki bu denge, verinin ne kadar kritik olduğuna göre bilinçli olarak seçilmelidir.

Performans izleme

Bir sorun, kullanıcılar şikâyet etmeden önce fark edilmelidir. Veritabanı performansı genellikle aniden değil, yavaş yavaş bozulur.

Örneğin bir sorgu, tablo küçükken 0,1 saniyede çalışıyordu. Tablo aylar içinde büyüdükçe sorgu süresi yavaş yavaş 5 saniyeye çıktı. İzleme yoksa bu bozulma, ancak uygulama yanıt veremez hâle geldiğinde fark edilir. İzleme

varsa; en yavaş sorgular, disk doluluk oranı, eşzamanlı bağlantı sayısı ve kilit beklemeleri önceden görülür ve sorun büyümeden çözülür.

Genel Ders

Veri hiçbir zaman kaybolmamalı, hiçbir zaman bozulmamalı ve her zaman erişilebilir olmalıdır. Bunun için yedekler düzenli olarak geri yüklenerek test edilmeli, replikasyonun insan hatasına karşı koruma sağlamadığı bilinmeli, yetkiler en az düzeyde tutulmalı, transaction'larla tutarlılık güvence altına alınmalı ve performans sürekli izlenmelidir.

Veri Mimarisi

Veri mimarisinin temel dersi şudur: tek tek araçlar değil, sistemin bütünü düşünülmelidir. Verinin nereden gelip nereye gittiği, kimin hangi veriden sorumlu olduğu, sistemin yıllar içinde nasıl büyüyeceği ve bunun ne kadara mal olacağı baştan planlanmalıdır.

Veri ambarı, veri gölü ve lakehouse

Veri ambarı (*data warehouse*), temizlenmiş, yapılandırılmış ve analiz için düzenlenmiş verinin tutulduğu sistemdir. Her şey önceden tanımlanmış tablolarda düzenli biçimde durur; hızlı ve güvenilir rapor alınır. Bedeli, içine nelerin konacağına ve nasıl düzenleneceğine önceden karar verilmesi gerekliliğidir.

Veri gölü (*data lake*), her türlü verinin ham hâliyle ve düşük maliyetle saklandığı sistemdir: kayıt dosyaları, görüntüler, JSON belgeleri, tablolar. Esnektir ve ucuzdur. Ancak düzen sağlanmazsa, zamanla kimsenin içinde ne olduğunu bilmediği ve hiçbir şeyin bulunamadığı bir yığına dönüşür; bu duruma veri bataklığı (*data swamp*) denir.

Lakehouse, ikisini birleştirmeyi amaçlar: veri gölü kadar ucuz ve esnek bir depolama üzerinde, veri ambarının düzenini ve güvenilirliğini sağlamak (Armbrust vd., 2021). Ders şudur: düzen ile esneklik arasında bir ödünleşim vardır ve her mimari bu ödünleşimin farklı bir noktasını seçer.

Veri modelleme yaklaşımları

Bir veri ambarının nasıl düzenleneceğine dair üç yerleşik yaklaşım vardır.

Kimball yaklaşımı, verinin insanların soracağı sorulara göre boyutsal modellerle düzenlenmesini önerir ve iş birimlerinin ihtiyaçlarından başlayarak aşağıdan yukarıya doğru inşa edilir (Kimball ve Ross, 2013). Hızlı sonuç verir ve satış raporlaması gibi işler için idealdir.

Inmon yaklaşımı, önce bütün kurumun verisinin tek, merkezî ve normalleştirilmiş bir yapıda toplanmasını, raporlara yönelik yapıların daha sonra bu merkezden türetilmesini önerir (Inmon, 2005). Daha tutarlı ve sağlam bir temel sunar, ancak kurulması çok daha uzun sürer.

Data Vault yaklaşımı ise verinin geçmişinin eksiksiz saklanmasını, hiçbir kaydın silinmemesini ve kaynak sistemlerdeki değişikliklere kolay uyum sağlanmasını öncelikli kılar (Linstedt ve Olschimke, 2015). Kaynak sistemleri sık değişen ve denetimin önemli olduğu bankacılık gibi alanlarda tercih edilir.

Ders şudur: bu yaklaşımlardan hiçbiri her durumda en iyi değildir; hız, tutarlılık ve değişime uyum arasında farklı dengeler kurarlar.

Data mesh

Data mesh, verinin tek bir merkezî ekip yerine, o veriyi üreten iş alanlarının kendileri tarafından yönetilmesini öneren bir yaklaşımdır (Dehghani, 2022). Her

ekip kendi verisini, başkalarının kullanabileceği temiz bir "ürün" olarak hazırlar; merkezî ekip ise yalnızca ortak altyapıyı ve kuralları sağlar.

Örneğin 500 kişilik bir kurumda bütün ekipler veri ihtiyaçları için tek bir merkezî veri ekibine istek gönderir ve aylarca sırada bekler. Merkezî ekip ise sipariş, ödeme ve müşteri verilerinin her birinin ayrıntılarını bilmediği için hatalar yapar. Data mesh yaklaşımında sipariş ekibi kendi sipariş verisinden, ödeme ekibi kendi ödeme verisinden sorumludur; çünkü veriyi en iyi tanıyan onlardır. Bu yaklaşımın riski ise her ekibin farklı standartlar kullanması ve verinin parçalanmasıdır; bu nedenle ortak kurallar zorunludur. Ders şudur: kurum büyüdükçe merkezî yapı darboğaz hâline gelir ve sorumluluğun dağıtılması gerekir.

Toplu ve gerçek zamanlı mimariler

Hem geçmiş verinin hem anlık verinin işlenmesi gereken sistemler için iki yerleşik mimari vardır.

Lambda mimarisinde iki ayrı yol bulunur (Marz ve Warren, 2015). Toplu işleme yolu, belirli aralıklarla bütün veriyi eksiksiz ve doğru biçimde yeniden hesaplar. Hız yolu ise gün içinde gelen veriyi anlık ama yaklaşık olarak işler. İki yolun sonuçları birleştirilerek sunulur. Bu mimarinin sorunu, aynı iş mantığının iki farklı sistemde iki kez yazılması ve bakımının iki kez yapılmasıdır; zamanla iki yolun sonuçları birbirinden ayrışabilir.

Kappa mimarisi bu ikiliği kaldırır (Kreps, 2014). Bütün veri tek bir akış işleme yolundan geçer. Geçmiş yeniden hesaplanmak istendiğinde, saklanmış olan akış baştan yeniden oynatılır. Ders şudur: iki ayrı sistemde aynı mantığı sürdürmek uzun vadede tutarsızlık üretir; mümkün olduğunda tek bir yol tercih edilmelidir.

Araç seçimi ve maliyet planlaması

Her durumda en iyi olan bir araç yoktur; ihtiyaca en uygun olan araç vardır. Aracın, problemin ölçeğine ve ekibin yetkinliğine uygun olması gerekir.

Örneğin günde birkaç gigabayt veri işleyen küçük bir şirketin büyük bir Spark kümesi kurması, hem gereksiz bir maliyet hem de gereksiz bir karmaşıklıktır; iyi yapılandırılmış tek bir PostgreSQL sunucusu bu ihtiyacı karşılar. Maliyet planlamasında ayrıca maliyetin büyümeyle birlikte nasıl değişeceği hesaplanmalıdır. Bugün ucuz görünen bir çözüm, veri yüz kat büyüdüğünde faturayı da yüz kat büyütebilir; başlangıçta pahalı görünen başka bir çözüm ise ölçek büyüdükçe daha ekonomik hâle gelebilir.

Ölçeklenebilir sistem tasarımı

Ölçeklenebilirlik (*scalability*), bir sistemin artan yükü baştan yeniden yazılmadan, yalnızca kaynak eklenerek karşılayabilmesidir. İki tür ölçekleme vardır: dikey ölçekleme (*vertical scaling*) mevcut makineyi daha güçlü bir makineyle değiştirmek, yatay ölçekleme (*horizontal scaling*) ise daha fazla makine eklemektir.

Örneğin tek bir büyük sunucuya dayanan bir sistem, o sunucunun sınırına geldiğinde yalnızca daha güçlü bir sunucu alarak büyüyebilir; bunun da fiziksel ve mali bir sınırı vardır. İşi baştan bağımsız parçalara bölünebilecek biçimde tasarlanmış bir sistem ise yük iki katına çıktığında iki kat makine ekleyerek çalışmaya devam eder.

Ancak bunun tersi de bir hatadır. Henüz ortaya çıkmamış bir ölçek için baştan aşırı karmaşık bir sistem kurmak, hem zaman hem para kaybıdır ve sistemi anlaşılmaz kılar. Knuth'un (1974) "erken optimizasyon" uyarısı burada da geçerlidir. Ders şudur: bugünün ihtiyacı basit biçimde çözülmeli, ancak yarın büyümenin önü kapatılmamalıdır.

Genel Ders

Veri mimarisi, düzen ile esneklik, merkezîlik ile dağıtıklık, basitlik ile ölçeklenebilirlik arasında bilinçli seçimler yapmaktır. Bugünün ihtiyacı basitçe çözülmeli, ama büyümeye açık kalınmalı; kimin hangi veriden sorumlu olduğu, verinin nasıl aktığı ve bunun büyüme ile birlikte ne kadara mal olacağı baştan bilinmelidir.

Veri Yönetişimi ve Veri Kalitesi

Veri yönetişimi (*data governance*), bir kurumdaki verinin sahipliğini, kullanım kurallarını, güvenliğini ve yasal uyumunu düzenleyen politika ve süreçler bütünüdür. Temel dersi şudur: veri bir varlıktır ve her varlık gibi sahibi, kuralları ve sorumlusu olmalıdır. Kimse neyin nerede olduğunu, ne anlama geldiğini ve kimin erişebileceğini bilmiyorsa, veri fayda değil risk üretir.

Veri kataloğu ve metadata yönetimi

Metadata, veri hakkındaki veridir: bir tablonun ne içerdiği, kimin sorumluluğunda olduğu, en son ne zaman güncellendiği, hangi kaynaktan geldiği. Veri kataloğu, bir kurumdaki bütün veri varlıklarının bu bilgilerle birlikte listelendiği ve arama yapılabilen envanterdir.

Örneğin yeni işe başlayan bir analist "müşteri iptal oranı" verisini arıyor. Katalog yoksa günlerce farklı kişilere sorar ve sonunda eskimiş ya da yanlış bir tabloyu kullanır. Katalog varsa, arama kutusuna yazar ve doğru tabloyu, tablonun açıklamasını, sorumlusunu ve en son güncellenme tarihini birkaç saniyede bulur.

Veri soyağacı

Veri soyağacı (*data lineage*), bir verinin hangi kaynaktan geldiğini, hangi dönüşüm adımlarından geçtiğini ve hangi raporlarda kullanıldığını gösteren kayıttır.

Soyağacı iki yönde işe yarar. Geriye doğru: yönetim raporundaki ciro bir gün aniden %20 düşmüş görünüyorsa, soyağacı bu rakamın üç ayrı tablodan hesaplandığını ve bunlardan birinin önceki gece hiç güncellenmediğini dakikalar içinde gösterir; soyağacı yoksa sorunun kaynağı saatlerce aranır. İleriye doğru: bir kaynak tabloda değişiklik yapılmadan önce, bu değişikliğin hangi raporları ve hangi modelleri etkileyeceği önceden görülür ve beklenmedik kırılmalar önlenir.

Veri kalitesi kuralları ve testleri

Veri kalitesi, verinin doğru, eksiksiz, tutarlı ve güncel olmasıdır. Kalite varsayılmamalı, sürekli ölçülmelidir. Great Expectations gibi araçlar, "bu sütun boş olamaz", "bu değer 0 ile 120 arasında olmalıdır", "bu tablodaki her müşteri numarası müşteri tablosunda bulunmalıdır" gibi kuralların tanımlanmasını ve her gün otomatik olarak kontrol edilmesini sağlar.

Örneğin bir müşteri tablosunda 1850 doğumlu müşteriler, telefon numarası alanında harfler, sipariş tablosunda gelecek tarihli siparişler bulunabilir. Bu hatalar tek tek önemsiz görünür; ancak raporlara ve modellere girdiklerinde ortalama yaşı, iletişim oranlarını ve satış tahminlerini sessizce bozar. Otomatik kurallar, bu tür veriyi raporlara ulaşmadan yakalar.

Erişim kontrolü ve veri gizliliği

Hassas veri yalnızca onu görmesi gereken kişiler tarafından görülmelidir. Bunun için erişim, kişinin rolüne göre sınırlandırılır; gerekmeyen hassas alanlar ise maskelenir ya da tamamen gizlenir.

Örneğin bir analist müşterilerin şehir bazında harcama alışkanlıklarını incelemek istiyor. Bu iş için müşterilerin kimlik numaralarını, telefon numaralarını ya da açık adreslerini görmesine gerek yoktur. Bu alanlar analistin erişiminden çıkarılır ya da maskelenir. Analist işini eksiksiz yapar, kişisel veri ise korunur. Bu yaklaşıma veri minimizasyonu (*data minimization*) denir: bir iş için yalnızca o işe gereken veri kullanılır.

KVKK ve GDPR uyumu

Kişisel veri işlemek yasal kurallara bağlıdır. Türkiye'de 6698 sayılı Kişisel Verilerin Korunması Kanunu (KVKK, 2016), Avrupa Birliği'nde ise Genel Veri Koruma Tüzüğü (GDPR; Avrupa Birliği, 2016) bu kuralları belirler. Her iki düzenleme de kişilere kendi verileri üzerinde haklar tanır ve kurumlara yükümlülükler getirir.

Bu düzenlemelerin iki temel ilkesi pratikte büyük önem taşır. Birincisi, kişinin verilerinin silinmesini talep etme hakkıdır (GDPR Madde 17; KVKK Madde 7 ve 11). Örneğin bir müşteri verilerinin silinmesini istediğinde kurum bunu yapmakla yükümlüdür. Ancak o müşterinin verisi on beş farklı sistemde, yedeklerde ve raporlarda dağınık hâlde duruyorsa, kurum verinin nerede olduğunu bile bulamaz. Veri kataloğu ve soyağacı burada doğrudan bir uyum aracına dönüşür. İkincisi, amaçla sınırlılık ilkesidir (GDPR Madde 5; KVKK Madde 4): veri, toplandığı amaç dışında kullanılmamalıdır. Örneğin kargo teslimatı için alınan bir telefon numarası, ayrı bir izin alınmadan pazarlama araması için kullanılamaz.

Uyumsuzluğun bedeli ağırdır. GDPR kapsamında en ağır ihlaller için öngörülen idari para cezası, 20 milyon avro ya da şirketin dünya çapındaki yıllık cirosunun %4'ü, hangisi yüksekse o kadar olabilir (Avrupa Birliği, 2016, Madde 83).

Ana veri yönetimi

Ana veri yönetimi (*master data management*, MDM), bir kurumun en temel varlıkları (müşteri, ürün, tedarikçi, çalışan) için tek ve doğru bir kayıt oluşturma ve bütün sistemlerin bu kaydı esas almasını sağlama sürecidir.

Örneğin aynı müşteri satış sisteminde "Ayşe Yılmaz", destek sisteminde "A. Yilmaz", faturalama sisteminde ise "Ayse YILMAZ" olarak kayıtlıdır. Kurum bunları üç ayrı müşteri olarak görür. Sonuç olarak müşteriye aynı kampanya e-postası üç kez gider, toplam harcaması yanlış hesaplanır, destek sistemindeki şikâyeti satış ekibi tarafından görülmez. Ana veri yönetimi, bu üç kaydı eşleştirip tek bir "altın kayıt" (*golden record*) altında birleştirir ve bütün sistemler o tek kaydı kullanır.

Genel Ders

Veri sahipsiz bırakılmamalıdır. Neyin nerede olduğu bilinmeli, nereden gelip nereye gittiği izlenmeli, doğruluğu sürekli test edilmeli, kimin göreceği kontrol edilmeli, yasal yükümlülükler gözetilmeli ve her temel varlığın tek bir doğru kaydı bulunmalıdır.

Veri Görselleştirme

Veri görselleştirmenin temel dersi şudur: grafik bir süs değil, bir iletişim aracıdır. İyi bir grafik, bir tablonun dakikalarca anlatamayacağını birkaç saniyede gösterir. Kötü bir grafik ise doğru veriyle bile yanlış bir şey anlatır.

Doğru grafik türünü seçmek

Her soru türünün ona uygun bir grafik türü vardır. Zaman içindeki değişim çizgi grafikle, kategoriler arası karşılaştırma çubuk grafikle, iki değişken arasındaki ilişki dağılım grafiğiyle (*scatter plot*) gösterilir.

Cleveland ve McGill (1984), insanların grafiklerdeki farklı görsel öğeleri ne kadar doğru algıladığını deneylerle ölçmüştür. Ortak bir eksen üzerindeki uzunlukları karşılaştırmak, açıları ya da alanları karşılaştırmaktan çok daha doğru sonuç vermektedir. Bu nedenle pasta grafik, yalnızca iki ya da üç parçalı bir bütünü göstermek için uygundur. On iki dilimli bir pasta grafikte hangi dilimin daha büyük olduğu göz tarafından ayırt edilemez; aynı veri çubuk grafikle gösterildiğinde fark hemen anlaşılır.

Renk, düzen ve okunabilirlik

Göz, bir grafikte önce rengi ve büyüklüğü fark eder. Bu durum bilinçli olarak kullanılmalıdır. Renk, dikkat çekilmesi gereken noktayı vurgulamak için kullanılmalı, süs için kullanılmamalıdır.

Örneğin on ülkenin satışlarını gösteren bir grafikte her çubuk farklı bir renge boyanırsa, göz hangi çubuğa bakacağını bilemez. Bütün çubuklar gri yapılıp yalnızca Türkiye turuncu yapılırsa, okuyucu doğrudan Türkiye'ye bakar.

Renk seçiminde erişilebilirlik de gözetilmelidir. Kırmızı-yeşil renk körlüğü, Kuzey Avrupa kökenli erkeklerin yaklaşık %8'inde görülür (Birch, 2012). Olumlu değerlerin yeşil, olumsuz değerlerin kırmızı gösterildiği bir grafik, bu kişiler için okunamaz hâle gelir.

Eksen seçimi de yanıltıcı olabilir. Bir çubuk grafikte dikey eksen sıfırdan değil 95'ten başlatılırsa, 96 ile 98 arasındaki küçük bir fark çok büyük görünür. Bu yöntem, istatistiklerin yanıltıcı sunumunun bilinen tekniklerinden biridir (Huff, 1954).

Veriyle hikâye anlatma

Bir grafiği gösterip "işte veri" demek yeterli değildir. Okuyucuya neye bakması gerektiği ve bundan ne sonuç çıkarıldığı söylenmelidir.

Örneğin başlığı "2026 Aylık Satışlar" olan bir grafik, okuyucuya hiçbir şey söylemez; okuyucu ne arayacağını kendisi bulmak zorundadır. Aynı grafiğin başlığı "Eylül'deki fiyat artışından sonra satışlar %18 düştü" olduğunda, okuyucu grafiğe bakmadan önce ne arayacağını bilir ve grafik bu iddiayı kanıtlayan bir delil hâline gelir. Ders şudur: grafiğin başlığı bir konu adı değil, bir sonuç cümlesi olmalıdır.

Araç seçimi

Matplotlib ve Seaborn, Python'da hızlı grafik çizmek için kullanılan kütüphanelerdir; Plotly ise etkileşimli grafikler üretir. Araç seçimi, grafiğin nerede kullanılacağına bağlıdır.

Analistin kendi keşif sürecinde veriye hızlıca bakması için basit ve hızlı araçlar yeterlidir; burada önemli olan hızdır. Sonuç bir yöneticiye ya da müşteriye sunulacaksa, üzerine gelindiğinde ayrıntı gösteren, yakınlaştırılabilen ve özenle biçimlendirilmiş grafikler daha uygundur; burada önemli olan anlaşılırlıktır. Ders şudur: keşif için çizilen grafik ile sunum için çizilen grafik farklı amaçlara hizmet eder.

Web tabanlı özel görselleştirme

D3.js, web tarayıcısında veriye dayalı görselleştirme yapmak için kullanılan bir JavaScript kütüphanesidir (Bostock, Ogievetsky ve Heer, 2011). Hazır kütüphaneler belirli grafik türleri sunarken, D3.js her görsel öğenin nereye çizileceğini, nasıl görüneceğini ve nasıl hareket edeceğini tek tek kontrol etmeye olanak tanır. Bu nedenle haber kuruluşlarının etkileşimli seçim haritaları ve veri haberleri gibi özel görselleştirmelerde yaygın olarak kullanılmıştır.

D3.js'in gücünün bedeli, öğrenmesinin ve yazmasının zaman almasıdır. Ders şudur: hazır grafik türleri ihtiyacı karşılıyorsa hazır araçlar kullanılmalı; özel araçlara ancak gerçekten özel bir anlatım gerektiğinde başvurulmalıdır.

Etkileşimli grafikler

Etkileşimli grafikler, okuyucunun kendi sorusunu sormasına izin verir. Statik bir grafik bir ülkenin ortalama işsizlik oranını gösterirken, etkileşimli bir grafikte okuyucu kendi şehrini seçebilir, yaş grubuna göre filtreleyebilir ve yıllar arasında gezinebilir.

Ancak her şeyi etkileşimli yapmak ana mesajı kaybettirebilir. Okuyucuların büyük kısmı hiçbir öğeye tıklamadan grafiğe bakıp geçer. Bu nedenle en önemli bulgu, hiçbir etkileşim yapılmadan da görülebilmelidir; etkileşim ayrıntı isteyenler için ek bir katman olmalıdır.

Genel Ders

Önce ne söylenmek istendiğine karar verilmeli, sonra bunu en açık gösteren grafik seçilmelidir. Renk dikkat çekmek için kullanılmalı, eksenler yanıltmamalı, grafiğin başlığı bir sonuç cümlesi olmalı ve ana mesaj hiçbir etkileşim gerektirmeden görülebilmelidir.

Ürün Analitiği

Ürün analitiğinin temel dersi şudur: kullanıcıların ne söylediğine değil, ne yaptığına bakılmalıdır. Bir ürünün iyi olup olmadığını anketlerden çok davranışlar gösterir.

Olay takibi

Olay takibi (*event tracking*), kullanıcıların bir üründe yaptığı eylemlerin kaydedilmesidir: bir sayfayı açmak, bir düğmeye tıklamak, bir ürünü sepete eklemek. Ölçülmeyen bir şey bilinemez; ancak her şeyi gelişigüzel ölçmek de kaosa yol açar.

Örneğin "kullanıcı düğmeye tıkladı" biçiminde bir kayıt hiçbir işe yaramaz. "Kullanıcı ödeme sayfasında Satın Al düğmesine tıkladı; sepet tutarı 450 TL, ürün sayısı 3" biçiminde bir kayıt ise analiz yapmaya olanak tanır. Olayların adlandırılması da baştan bir standarda bağlanmalıdır. Bir geliştirici olayı `purchase`, ikincisi `satin_alma`, üçüncüsü `order_completed` olarak adlandırırsa, aynı olay üç farklı isimle dağılır ve toplam satın alma sayısı doğru hesaplanamaz.

Huni analizi

Huni analizi (*funnel analysis*), kullanıcıların bir hedefe ulaşırken geçtikleri adımların her birinde ne kadarının kaybolduğunu gösterir.

Örneğin bir e-ticaret sitesinde 10.000 kişi ürün sayfasına gelir, 3.000'i ürünü sepete ekler, 2.500'ü ödeme sayfasına geçer ve yalnızca 500'ü satın almayı tamamlar. En büyük kayıp, 2.500 kişinin 2.000'inin ayrıldığı ödeme sayfasındadır. Bu durumda sorun büyük olasılıkla reklamda ya da üründe değil, ödeme sayfasının kendisindedir: form çok uzun olabilir, sayfa güven vermiyor olabilir ya da kargo ücreti yalnızca son adımda gösteriliyor olabilir. Huni analizi, iyileştirme çabasının en çok kayıp yaşanan adıma yönlendirilmesini sağlar.

Kohort ve elde tutma analizi

Kohort (*cohort*), aynı dönemde ürüne katılan kullanıcılardan oluşan gruptur. Kohort analizi, bu grupların zaman içinde ne kadarının ürünü kullanmaya devam ettiğini (elde tutma, *retention*) karşılaştırır.

Örneğin Ocak ayında gelen kullanıcıların %40'ı bir ay sonra hâlâ ürünü kullanırken, Mart ayında gelenlerin yalnızca %15'i kullanmaya devam etmektedir. Bu sırada toplam kullanıcı sayısı artıyor olabilir, çünkü reklamlarla sürekli yeni kullanıcı gelmektedir. Toplam rakam olumlu bir tablo çizer; kohort analizi ise yeni gelen kullanıcıların eskiler kadar memnun kalmadığını ve belki Mart ayında yapılan bir değişikliğin deneyimi bozduğunu ortaya çıkarır. Ders şudur: toplam rakamlar, altındaki bozulmayı gizleyebilir.

A/B testleri ve istatistiksel anlamlılık

A/B testi, bir değişikliğin etkisini, kullanıcıların rastgele iki gruba ayrılması ve yalnızca birine değişikliğin uygulanmasıyla ölçen kontrollü deneydir. Ölçülen farkın gerçek mi yoksa rastlantısal mı olduğunu anlamak için istatistiksel anlamlılık değerlendirilir.

Örneğin yeni bir tasarım 100 kişiye gösterilmiş ve 12'si satın almıştır; eski tasarım 100 kişiye gösterilmiş ve 10'u satın almıştır. Bu kadar küçük bir örneklemde iki puanlık fark tamamen şans eseri olabilir; yeni tasarımın daha iyi olduğu söylenemez. Aynı fark binlerce kullanıcıyla yapılan bir testte de korunuyorsa, ona güvenilebilir.

A/B testlerinde sık yapılan bir hata, testin sonuçlarının her gün kontrol edilmesi ve sonuç ilk kez anlamlı göründüğü gün testin durdurulmasıdır. Sonuçlar rastlantısal olarak dalgalandığı için bu yöntem, aslında etkisi olmayan bir değişikliği etkili gibi gösterme olasılığını ciddi biçimde artırır (Johari vd., 2017). Bu nedenle gereken örneklem büyüklüğü testten önce belirlenmeli ve test bu büyüklüğe ulaşmadan durdurulmamalıdır.

Analitik araçları

Google Analytics, Mixpanel ve Amplitude aynı temel işi yapar, ancak odakları farklıdır. Google Analytics daha çok "kullanıcılar siteye nereden geliyor ve hangi sayfalara bakıyor?" sorusuna cevap verir; pazarlama ve trafik analizinde güçlüdür. Mixpanel ve Amplitude ise "kullanıcı ürünün içinde ne yapıyor, hangi özelliği kullanıyor, kim geri dönüyor?" sorularına odaklanır; ürün ekipleri için daha uygundur. Ders şudur: araç, sorulacak sorunun türüne göre seçilir.

Ürün metrikleri

Ürün analitiğinde az sayıda ama doğru metrik takip edilmeli ve bu metriklerin birbiriyle ilişkisi anlaşılmalıdır. Başlıca metrikler şunlardır:

- Günlük aktif kullanıcı (*daily active users*, DAU): ürünün ne kadar canlı olduğunu gösterir.
- Kayıp oranı (*churn*): belirli bir dönemde ürünü bırakan kullanıcıların oranını gösterir.
- Müşteri yaşam boyu değeri (*lifetime value*, LTV): bir kullanıcının ürünle ilişkisi boyunca kuruma getirdiği toplam geliri gösterir.
- Müşteri edinme maliyeti (*customer acquisition cost*, CAC): bir kullanıcıyı kazanmak için harcanan ortalama tutarı gösterir.

Bu metrikler birlikte okunduğunda anlam kazanır. Örneğin bir kullanıcıyı reklamla kazanmak 200 TL'ye mal oluyor ama o kullanıcının yaşam boyu değeri 150 TL ise, kurum her yeni kullanıcıda 50 TL kaybetmektedir. Bu durumda büyümek kârı artırmaz, zararı büyütür. Kullanıcı sayısındaki hızlı artış, tek başına bakıldığında başarı gibi görünür.

Genel Ders

Kullanıcının yolculuğu adım adım ölçülmeli, en çok nerede kaybolduğu bulunmalı, yeni gelenlerin eskiler kadar kalıp kalmadığı izlenmeli, değişiklikler tahminle değil kontrollü deneyle doğrulanmalı ve bir kullanıcıyı kazanmanın maliyetinin onu elde tutmanın değerinden düşük olduğundan emin olunmalıdır.

Zaman Serileri ve Tahmin

Zaman serisi (*time series*), belirli zaman aralıklarıyla sırayla kaydedilmiş gözlemlerden oluşan veridir: günlük satışlar, saatlik elektrik tüketimi, aylık işsizlik oranı. Temel ders şudur: geçmiş, geleceğe dair ipucu verir ama garanti vermez. Zaman serilerinde sıra önemlidir; dün bugünü, bugün yarını etkiler. Bir tahminin değeri, ne kadar emin olunduğunun da söylenebilmesiyle ortaya çıkar.

Trend, mevsimsellik ve gürültü

Her zaman serisi üç bileşenden oluşur. Trend, uzun vadeli yöndür. Mevsimsellik, belirli aralıklarla tekrar eden döngülerdir. Gürültü ise geriye kalan rastlantısal oynamadır. Bu bileşenler birbirinden ayrılmadan doğru yorum yapılamaz (Hyndman ve Athanasopoulos, 2021).

Örneğin bir dondurmacının satışları Temmuz'da Ocak'a göre beş kat fazladır. Bu bir büyüme değil, mevsimselliktir. Asıl sorulması gereken soru, bu Temmuz'un geçen Temmuz'dan daha iyi olup olmadığıdır; bu da trendi gösterir. Yağmurlu bir günde satışların düşmesi ise gürültüdür. Mevsimselliği büyüme sanmak ya da gürültüye bakarak karar vermek, zaman serilerinde en sık yapılan hatalardır.

Hareketli ortalamalar

Hareketli ortalama, her nokta için son belirli sayıda gözlemin ortalamasının alınmasıdır. Günlük dalgalanmayı yumuşatır ve asıl yönü görünür kılar.

Örneğin bir web sitesinin günlük ziyaretçi sayısı hafta sonları düşüp hafta içi yükselir ve günlük grafik testere dişi gibi zikzaklı görünür. 7 günlük hareketli ortalama alındığında haftalık döngü kaybolur ve sitenin genel olarak büyüyüp büyümediği netleşir. Ancak hareketli ortalama geçmiş değerlere dayandığı için yön değişimlerini her zaman bir miktar gecikmeyle gösterir. Ani bir düşüş, hareketli ortalamada birkaç gün sonra belirginleşir.

ARIMA ve SARIMA

ARIMA (*AutoRegressive Integrated Moving Average*), bir serinin gelecekteki değerini kendi geçmiş değerlerinden ve geçmiş tahmin hatalarından hesaplayan klasik bir yöntemdir (Box ve Jenkins, 1970). SARIMA, bu modele mevsimsel bileşeni ekler.

Örneğin bir şehrin aylık elektrik tüketimi tahmin edilmek istensin. Geçen ayın tüketimi bu ayı etkiler; geçen yılın aynı ayı da etkiler, çünkü kışın ısınma, yazın soğutma ihtiyacı her yıl tekrar eder. SARIMA bu iki etkiyi birlikte hesaba katar. Bu yöntemlerin avantajı açıklanabilir olmalarıdır; modelin her bileşeninin ne yaptığı adım adım anlaşılabilir. Dezavantajı ise ani kırılmalarda ve çok sayıda değişkenin karmaşık biçimde etkileştiği durumlarda zorlanmalarıdır.

Prophet

Prophet, Meta (eski adıyla Facebook) tarafından geliştirilen ve iş dünyasındaki pratik tahmin ihtiyaçları için tasarlanmış bir araçtır (Taylor ve Letham, 2018). Trend, mevsimsellik ve özel günlerin etkisini ayrı bileşenler olarak modeller ve istatistik uzmanı olmayan kullanıcıların da makul tahminler üretmesini hedefler.

Örneğin bir e-ticaret sitesinin satışları Kurban Bayramı'ndan önce artar. Ancak bayram hicri takvime göre belirlendiği için miladi takvimde her yıl yaklaşık 11 gün öne kayar; dolayısıyla "her yıl Haziran" gibi sabit bir mevsimsellik bu etkiyi yakalayamaz. Prophet'e bayram tarihleri ayrıca verilir ve bu etkiyi ayrı olarak öğrenir.

Derin öğrenme modelleri

LSTM (*Long Short-Term Memory*), uzun diziler boyunca bilgiyi hatırlayabilen bir yapay sinir ağı türüdür (Hochreiter ve Schmidhuber, 1997). Çok fazla veri ve karmaşık ilişkiler bulunan zaman serilerinde örüntüleri kendisi öğrenebilir.

Örneğin bir elektrik dağıtım şirketi binlerce bölgenin saatlik tüketimini; hava durumu, tatil günleri ve geçmiş tüketim verileriyle birlikte tahmin etmek istediğinde, klasik yöntemler bu karmaşıklık karşısında yetersiz kalabilir. Ancak büyük tahmin yarışmalarının sonuçları, birçok durumda basit istatistiksel yöntemlerin karmaşık makine öğrenmesi yöntemleriyle yarışabildiğini, hatta onları geçebildiğini göstermiştir (Makridakis, Spiliotis ve Assimakopoulos, 2018). Ders şudur: karmaşık model her zaman daha iyi tahmin anlamına gelmez; basit yöntemle başlanmalı, karmaşığa ancak gerekirse geçilmelidir.

Tahmin doğruluğunu ölçmek

Bir tahmin yönteminin değerlendirilmesinde iki temel kural vardır: gelecekteki veri, geçmişi tahmin etmek için kullanılmamalı ve model basit bir rakiple karşılaştırılmalıdır.

Sıradan makine öğrenmesi problemlerinde veri rastgele karıştırılıp eğitim ve test kümelerine ayrılır. Zaman serisinde bu yapılırsa model, gelecekteki noktaları görerek geçmişteki noktaları tahmin eder ve olduğundan çok daha başarılı görünür. Doğru yöntem zamana göre ayırmaktır: örneğin 2024 sonuna kadarki veriyle eğitilen model, 2025 verisi üzerinde test edilir (Hyndman ve Athanasopoulos, 2021).

İkinci kural, modelin en basit tahmine karşı sınanmasıdır. En basit tahmin, "yarın bugünle aynı olacak" ya da "bu Temmuz geçen Temmuz'la aynı olacak" diyen saf tahmindir (*naive forecast*). Karmaşık bir model, bu saf tahminden belirgin biçimde daha iyi değilse, karmaşıklığı gereksizdir. Şaşırtıcı ölçüde sık olarak daha iyi değildir.

Anomali tespiti

Anomali tespiti (*anomaly detection*), beklenen davranıştan belirgin biçimde sapan gözlemlerin bulunmasıdır. Bir şeyin anormal olduğunu söyleyebilmek için

önce neyin normal olduğunun bilinmesi gerekir.

Örneğin bir sunucunun işlemci kullanımı gece saat üçte %90'a çıksın. Bu bir saldırı olabilir; ancak her gece aynı saatte çalışan bir yedekleme işi de olabilir. Model günlük olağan örüntüyü öğrendiyse, gece üçteki yükselişi normal sayar. Aynı yükseliş öğleden sonra saat ikide gerçekleşirse, bu olağan örüntünün dışında kaldığı için alarm verir. Bankalar da aynı mantıkla, bir müşterinin alışkanlıklarının dışındaki bir kart işlemini fark eder ve müşteriyle iletişime geçer.

Genel Ders

Önce seri trend, mevsimsellik ve gürültü olarak ayrıştırılmalıdır. Basit yöntemle başlanmalı, karmaşık yönteme ancak gerekirse geçilmelidir. Tahmin her zaman geçmişten geleceğe doğru test edilmeli ve en basit tahminle karşılaştırılmalıdır. Hiçbir model, geçmişte hiç yaşanmamış bir olayı öngöremez; tahminler bu sınırla birlikte okunmalıdır.

Coğrafi Veri

Coğrafi veri (*geospatial data*), bir konum bilgisi taşıyan veridir: bir mağazanın adresi, bir aracın güzergâhı, bir ilçenin sınırı, bir bölgenin sıcaklığı. Temel ders şudur: konum, verinin en güçlü bağlamlarından biridir. Ancak dünya yuvarlak, haritalar düzdür. Bu nedenle coğrafi veriyle hesap yapmadan önce verinin hangi koordinat sisteminde olduğu bilinmelidir; aksi hâlde doğru görünen yanlış sonuçlar ortaya çıkar.

Koordinat sistemleri ve projeksiyonlar

Koordinat sistemi, bir noktanın dünya üzerindeki yerini sayılarla ifade etme yöntemidir. **Projeksiyon** ise yuvarlak dünyanın düz bir yüzeye aktarılma yöntemidir. Yuvarlak bir yüzeyi düzleştirmenin kusursuz bir yolu yoktur; her projeksiyon bazı özellikleri (alan, açı, mesafe, yön) korurken diğerlerini bozar (Snyder, 1987).

Örneğin web haritalarında yaygın olarak kullanılan Mercator projeksiyonu yönleri ve yerel şekilleri korur, ancak kutuplara yaklaştıkça alanları büyük ölçüde büyütür. Bu projeksiyonda Grönland neredeyse Afrika kadar büyük görünür; oysa Afrika'nın yüz ölçümü yaklaşık 30 milyon km^2, Grönland'ınki ise yaklaşık 2,2 milyon km^2'dir. Yani Afrika Grönland'dan yaklaşık 14 kat büyüktür. Mercator yön bulmak için iyidir, alan karşılaştırmak için kötüdür.

Pratikteki en yaygın tuzak birimdir. GPS cihazlarının verdiği enlem ve boylam değerleri derece cinsindendir, metre cinsinden değildir. Üstelik bir boylam derecesinin metre karşılığı ekvatordan kutuplara doğru küçülür. İki nokta arasındaki koordinat farkı doğrudan hesaplanırsa anlamsız bir sayı elde edilir. Mesafe ya da alan ölçmek için veri önce metre kullanan uygun bir projeksiyona dönüştürülmeli ya da küre üzerinde mesafe hesaplayan yöntemler kullanılmalıdır.

Vektör ve raster veri

Coğrafi veri iki temel biçimde saklanır. Vektör veri, nesneleri geometrik şekillerle ifade eder: bir okulun konumu nokta, bir yol çizgi, bir ilçe sınırı çokgendir. Net sınırları olan nesneler için kullanılır. Raster veri ise alanı eşit büyüklükte hücrelerden oluşan bir ızgaraya böler ve her hücreye bir değer atar: uydu görüntüleri, sıcaklık haritaları, yükseklik haritaları. Her yere yayılan ve sürekli değişen olgular için kullanılır.

Örneğin "bu mahallede kaç eczane var?" sorusu vektör veriyle, "bu bölgenin ortalama yüzey sıcaklığı nedir?" sorusu raster veriyle yanıtlanır. Ders şudur: veri türü, temsil edilen olgunun doğasına göre seçilir.

Mekânsal sorgular

PostGIS, PostgreSQL veritabanına coğrafi veri türleri ve mekânsal sorgu yeteneği ekleyen bir uzantıdır. Bu sayede veritabanına "nerede" sorusu doğrudan soru-

labilir.

Örneğin bir kargo şirketi "bu müşterinin adresine 5 km içindeki en yakın üç şube hangileridir?" ya da "hangi depolar deprem riski yüksek bölgelerin içinde kalıyor?" sorularını doğrudan SQL ile sorabilir. Sıradan bir veritabanı bu soruları anlayamaz. Burada da index önemlidir: mekânsal index olmadan milyonlarca nokta arasında yakınlık araması her nokta tek tek kontrol edildiği için çok yavaşlar; mekânsal index ise yalnızca ilgili bölgedeki noktaların kontrol edilmesini sağlar.

Katmanlarla çalışma

QGIS, coğrafi veriyle kod yazmadan çalışmaya olanak tanıyan açık kaynaklı bir coğrafi bilgi sistemi (*geographic information system*, GIS) yazılımıdır. Temel çalışma biçimi, farklı veri katmanlarını üst üste koyarak aralarındaki ilişkiyi görmektir.

Örneğin bir belediye planlamacısı nüfus yoğunluğu katmanını, mevcut park alanları katmanını ve boş arsalar katmanını üst üste koyar. Nüfusun yoğun, parkın az olduğu bölgelerdeki boş arsalar, yeni park için en uygun adaylar olarak doğrudan görünür hâle gelir. Ders şudur: bazı sorular için en hızlı çözüm yolu, veriyi harita üzerinde görmektir.

Konuma göre birleştirme

GeoPandas, Python'da tablo biçimindeki veriye geometri sütunu ekleyerek coğrafi veriyle çalışmayı sağlayan bir kütüphanedir. Coğrafi veride en önemli işlemlerden biri, iki tablonun ortak bir sütun yerine konum üzerinden birleştirilmesidir. Bu işleme mekânsal birleştirme (*spatial join*) denir.

Örneğin elde binlerce müşteri adresinin koordinatları ve Türkiye'nin ilçe sınırlarını gösteren çokgenler bulunsun. Müşteri tablosunda ilçe bilgisi yoktur. Mekânsal birleştirme, her müşteri noktasının hangi ilçe çokgeninin içine düştüğünü bulur ve müşteriye ilçe bilgisini ekler. Ardından ilçe bazında müşteri sayısı ya da ortalama harcama hesaplanabilir.

Harita görselleştirme

Folium ve Kepler.gl gibi araçlar, konumlu veriyi etkileşimli haritalar üzerinde göstermeye olanak tanır. Örneğin bir şehirdeki milyonlarca taksi yolculuğunu tablo hâlinde incelemek mümkün değildir. Bu yolculuklar bir ısı haritası olarak gösterildiğinde, sabah saatlerinde hangi semtlerden kalkılıp hangi iş merkezlerine gidildiği bir bakışta görülür.

Ancak harita görselleştirmede çok yaygın bir yanılgı vardır: birçok harita, aslında yalnızca nüfus yoğunluğunu gösterir. Kalabalık bölgelerde her şey daha fazladır: müşteri, suç, kahve dükkânı, trafik kazası. Bir haritada "müşterilerimiz en çok şehir merkezinde" görüldüğünde, bu yalnızca insanların en çok şehir merkezinde yaşadığını gösteriyor olabilir.[2] Gerçek bir örüntü görmek için sayılar

[2]Bu yanılgı, Munroe (2012) tarafından bir karikatürle popüler biçimde ifade edilmiştir.

nüfusa ya da başka bir uygun tabana oranlanmalıdır: toplam müşteri sayısı yerine bin kişi başına müşteri sayısı gibi.

Mesafe, alan ve yakınlık analizleri

"Yakın" kavramının anlamı bağlama göre değişir. Kuş uçuşu mesafe (*Euclidean distance*), iki nokta arasındaki düz çizgidir. Ağ mesafesi (*network distance*) ise gerçek yollar üzerinden ölçülen mesafe ya da süredir.

Örneğin bir haritada iki nokta kuş uçuşu 2 km uzaklıkta görünebilir. Ancak aralarında köprüsü olmayan bir nehir ya da yaya geçidi bulunmayan bir otoyol varsa, yürüyerek gitmek kırk dakika sürebilir. Bir hastaneye ya da okula erişim analiz ediliyorsa, kuş uçuşu mesafe değil, yol ağı üzerinden ulaşım süresi kullanılmalıdır. Aynı şekilde bir ilçenin alanı alanı korumayan bir projeksiyonda hesaplanırsa, sonuç yüzde onlarca hatalı çıkabilir.

Genel Ders

Coğrafi veriyle çalışmadan önce verinin hangi koordinat sisteminde olduğu ve hangi birimi kullandığı kontrol edilmelidir. Nesneler için vektör, sürekli olgular için raster veri kullanılmalıdır. "Yakın" denildiğinde kuş uçuşu mesafenin mi yoksa gerçek ulaşımın mı kastedildiği açıkça belirlenmelidir. Haritada görülen bir örüntünün yalnızca nüfus yoğunluğunun yansıması olup olmadığı sorgulanmalıdır.

Sonuç

Farklı veri alanlarından çıkan dersler, birkaç ortak ilkede birleşir.

Birincisi, veri bir temsildir. Ham sayılar, onları üreten süreç (kim ölçtü, neyi ölçmedi, hangi birimle ölçtü) bilinmeden yorumlanamaz. Hayatta kalan yanlılığı, karıştırıcı değişkenler, Simpson paradoksu, birim hataları ve yalnızca nüfus yoğunluğunu gösteren haritalar, aynı hatanın farklı alanlardaki görünümleridir: temsil ile gerçekliğin karıştırılması.

İkincisi, zaman her şeyi eskitir. Modeller dağılım değişimiyle, veri tanımları kurumsal değişimle, veritabanı performansı veri büyümesiyle bozulur. Bu nedenle sürümleme, izleme ve geri dönüş yeteneği; makine öğrenmesinde, analitik mühendisliğinde ve veritabanı yönetiminde aynı ihtiyaca cevap verir. Bir sistemin bugün çalışıyor olması, yarın da çalışacağının kanıtı değildir.

Üçüncüsü, güven kurulan bir şeydir. Bir rakama güvenilmesi; otomatik testlerle, açık tanımlarla, verinin kökeninin izlenmesiyle ve her temel varlık için tek bir doğru kayıtla sağlanır. Veri kalitesinin sorumluluğu yalnızca bir ekibe bırakılamaz; veriyi üreten, dönüştüren ve sunan her aşamanın parçasıdır.

Dördüncüsü, basitlik bir varsayılandır. Netflix ödülü, tahmin yarışmalarının sonuçları ve küçük ölçekli sistemler için kurulan gereksiz büyük veri altyapıları aynı dersi verir: karmaşıklık kendi maliyetini taşır ve ancak gerçek bir ihtiyaç kanıtlandığında eklenmelidir. Her alanda en basit çözüm önce denenmeli ve karmaşık çözüm ona karşı sınanmalıdır.

Beşincisi, ölçülen şey davranışı biçimlendirir. Yanlış KPI, yanlış başarı metriği ya da yanlış optimizasyon hedefi, sistemi ve insanları asıl amaçtan uzaklaştırır. Ölçek büyüdükçe ve bulutta her kaynak faturalandırıldıkça, verimlilik de teknik bir tercih olmaktan çıkıp ekonomik bir zorunluluğa dönüşür.

Altıncısı, veri insanlarla ilgilidir. Geçmiş veriden öğrenen sistemler geçmişin adaletsizliklerini sürdürebilir, geri besleme döngüleri bu adaletsizlikleri büyütebilir ve kişisel veri, yasal ve etik sorumluluk doğurur. Teknik doğruluk, adaletin ve hukuka uygunluğun garantisi değildir.

Bu çerçevenin sınırları da vardır. Burada aktarılan dersler genel eğilimlerdir; her biri belirli bağlamlarda istisna gösterir. Örneğin bazı alanlarda karmaşık modeller basit modellerden açık ara üstündür ve bazı sistemler baştan büyük ölçek için tasarlanmayı gerçekten gerektirir. Dersler, kararları otomatikleştiren kurallar değil, kararları sorgulamaya yarayan sorulardır.

Açık kalan bir soru, yapay zekâ destekli araçların veri işinin teknik kısmını giderek daha fazla üstlenmesinin bu ilkelerin önemini nasıl değiştireceğidir. Kod yazmak ve araç kullanmak kolaylaştıkça, doğru soruyu sormak, yanlılığı fark etmek, doğru metriği seçmek ve sonucun sınırlarını bilmek, insanın katkısının daha büyük bir kısmını oluşturacaktır. Hızla üretilen analizlerin her biri yukarıdaki tuzaklara aynı hızla düşebileceği için, bu ilkeler araçlar geliştikçe önemsizleşmek yerine daha belirleyici hâle gelebilir.

Kaynaklar

- Amatriain, X. ve Basilico, J. (2012). Netflix Recommendations: Beyond the 5 Stars. *Netflix Technology Blog*.
- Armbrust, M. vd. (2020). Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores. *Proceedings of the VLDB Endowment*, 13(12).
- Armbrust, M., Ghodsi, A., Xin, R. ve Zaharia, M. (2021). Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics. *CIDR 2021*.
- Avrupa Birliği (2016). Regulation (EU) 2016/679 (General Data Protection Regulation). *Official Journal of the European Union*.
- Berenson, H. vd. (1995). A Critique of ANSI SQL Isolation Levels. *ACM SIGMOD*.
- Bergstra, J. ve Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 13.
- Birch, J. (2012). Worldwide Prevalence of Red-Green Color Deficiency. *Journal of the Optical Society of America A*, 29(3).
- Bostock, M., Ogievetsky, V. ve Heer, J. (2011). D^3: Data-Driven Documents. *IEEE Transactions on Visualization and Computer Graphics*, 17(12).
- Box, G. E. P. (1976). Science and Statistics. *Journal of the American Statistical Association*, 71(356).
- Box, G. E. P. ve Draper, N. R. (1987). *Empirical Model-Building and Response Surfaces*. Wiley.
- Box, G. E. P. ve Jenkins, G. M. (1970). *Time Series Analysis: Forecasting and Control*. Holden-Day.
- Caruana, R. vd. (2015). Intelligible Models for HealthCare: Predicting Pneumonia Risk and Hospital 30-day Readmission. *ACM KDD*.
- Charig, C. R., Webb, D. R., Payne, S. R. ve Wickham, J. E. A. (1986). Comparison of Treatment of Renal Calculi by Open Surgery, Percutaneous Nephrolithotomy, and Extracorporeal Shockwave Lithotripsy. *British Medical Journal*, 292.
- Cleveland, W. S. ve McGill, R. (1984). Graphical Perception: Theory, Experimentation, and Application to the Development of Graphical Methods. *Journal of the American Statistical Association*, 79(387).
- Dageville, B. vd. (2016). The Snowflake Elastic Data Warehouse. *ACM SIGMOD*.

- Dastin, J. (2018). Amazon Scraps Secret AI Recruiting Tool that Showed Bias against Women. *Reuters*.
- Dehghani, Z. (2022). *Data Mesh: Delivering Data-Driven Value at Scale*. O'Reilly.
- Ensign, D., Friedler, S. A., Neville, S., Scheidegger, C. ve Venkatasubramanian, S. (2018). Runaway Feedback Loops in Predictive Policing. *Proceedings of Machine Learning Research*, 81 (FAT* 2018).
- Galton, F. (1886). Regression Towards Mediocrity in Hereditary Stature. *Journal of the Anthropological Institute of Great Britain and Ireland*, 15.
- Goodhart, C. A. E. (1975). Problems of Monetary Management: The U.K. Experience. *Papers in Monetary Economics*, Reserve Bank of Australia.
- Härder, T. ve Reuter, A. (1983). Principles of Transaction-Oriented Database Recovery. *ACM Computing Surveys*, 15(4).
- Hochreiter, S. ve Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8).
- Huff, D. (1954). *How to Lie with Statistics*. W. W. Norton.
- Hyndman, R. J. ve Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3. baskı). OTexts.
- Inmon, W. H. (2005). *Building the Data Warehouse* (4. baskı). Wiley.
- Johari, R., Koomen, P., Pekelis, L. ve Walsh, D. (2017). Peeking at A/B Tests: Why It Matters, and What to Do About It. *ACM KDD*.
- Kahneman, D. (2011). *Thinking, Fast and Slow*. Farrar, Straus and Giroux.
- Kaufman, S., Rosset, S., Perlich, C. ve Stitelman, O. (2012). Leakage in Data Mining: Formulation, Detection, and Avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4).
- Kimball, R. ve Ross, M. (2013). *The Data Warehouse Toolkit* (3. baskı). Wiley.
- Knuth, D. E. (1974). Structured Programming with go to Statements. *ACM Computing Surveys*, 6(4).
- Kreps, J. (2014). Questioning the Lambda Architecture. *O'Reilly Radar*.
- Kreps, J., Narkhede, N. ve Rao, J. (2011). Kafka: A Distributed Messaging System for Log Processing. *NetDB Workshop*.
- KVKK (2016). 6698 sayılı Kişisel Verilerin Korunması Kanunu. *Resmî Gazete*, 29677.

- Linstedt, D. ve Olschimke, M. (2015). *Building a Scalable Data Warehouse with Data Vault 2.0*. Morgan Kaufmann.
- Makridakis, S., Spiliotis, E. ve Assimakopoulos, V. (2018). Statistical and Machine Learning Forecasting Methods: Concerns and Ways Forward. *PLOS ONE*, 13(3).
- Mangel, M. ve Samaniego, F. J. (1984). Abraham Wald's Work on Aircraft Survivability. *Journal of the American Statistical Association*, 79(386).
- Marz, N. ve Warren, J. (2015). *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning.
- Munroe, R. (2012). Heatmap. *xkcd*, 1138.
- NASA (1999). *Mars Climate Orbiter Mishap Investigation Board Phase I Report*.
- Nightingale, F. (1858). *Notes on Matters Affecting the Health, Efficiency, and Hospital Administration of the British Army*. Harrison and Sons.
- Open Science Collaboration (2015). Estimating the Reproducibility of Psychological Science. *Science*, 349(6251).
- PostgreSQL Global Development Group (2024). *PostgreSQL Documentation: Resource Consumption*.
- Saltzer, J. H. ve Schroeder, M. D. (1975). The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9).
- Sculley, D. vd. (2015). Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems (NeurIPS)*.
- Shvachko, K., Kuang, H., Radia, S. ve Chansler, R. (2010). The Hadoop Distributed File System. *IEEE MSST*.
- Siebert, H. (2001). *Der Kobra-Effekt*. Deutsche Verlags-Anstalt.
- Simmons, J. P., Nelson, L. D. ve Simonsohn, U. (2011). False-Positive Psychology. *Psychological Science*, 22(11).
- Simpson, E. H. (1951). The Interpretation of Interaction in Contingency Tables. *Journal of the Royal Statistical Society, Series B*, 13(2).
- Snyder, J. P. (1987). *Map Projections: A Working Manual*. U.S. Geological Survey Professional Paper 1395.
- Strathern, M. (1997). 'Improving Ratings': Audit in the British University System. *European Review*, 5(3).
- Taylor, S. J. ve Letham, B. (2018). Forecasting at Scale. *The American Statistician*, 72(1).

- Wachter, S., Mittelstadt, B. ve Floridi, L. (2017). Why a Right to Explanation of Automated Decision-Making Does Not Exist in the General Data Protection Regulation. *International Data Privacy Law*, 7(2).
- Wolpert, D. H. (1996). The Lack of A Priori Distinctions Between Learning Algorithms. *Neural Computation*, 8(7).
- Zaharia, M. vd. (2012). Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. *USENIX NSDI*.